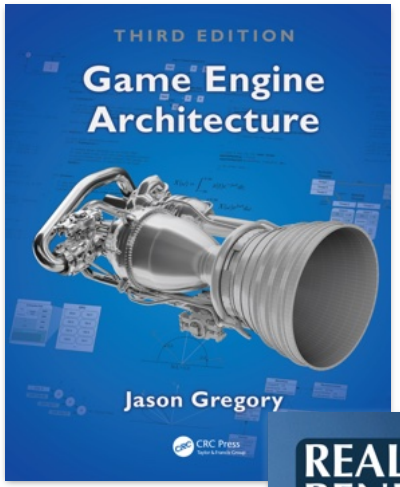
Lecture 9

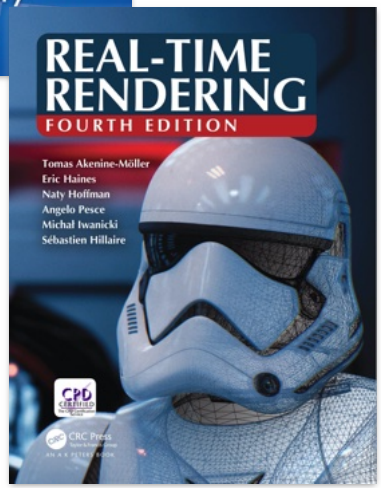
Fernando Birra
Rui Nóbrega

References

- Jason Gregory, **Game Engine Architecture 3rd Ed.**, CRC Press, 2018.

- Tomas Akenine-Möller et al; **Real-time Rendering**, CRC Press, 2018.





Graphics Engine Architecture and Pipeline

- Game Engine Runtime Architecture
- Graphics Rendering Pipeline
- Pipeline Optimization

Game Engine Runtime Architecture

Fernando Birra
Rui Nóbrega

Game Engine

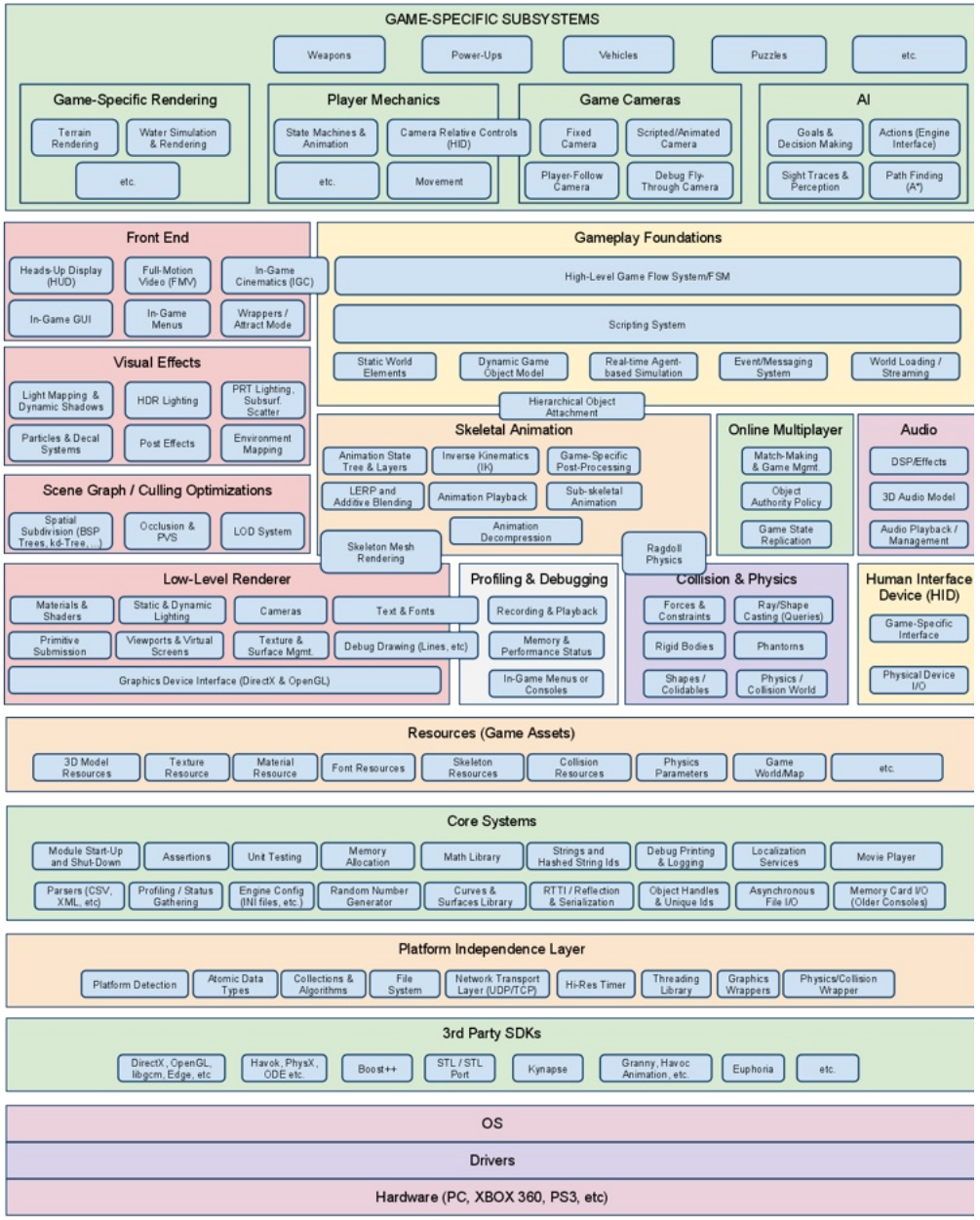
- Usually composed of:
  - A set of tools (tool suite)
  - A runtime component

game specific layer

The runtime component of a game engine is a highly complex piece of software built using a layered approach

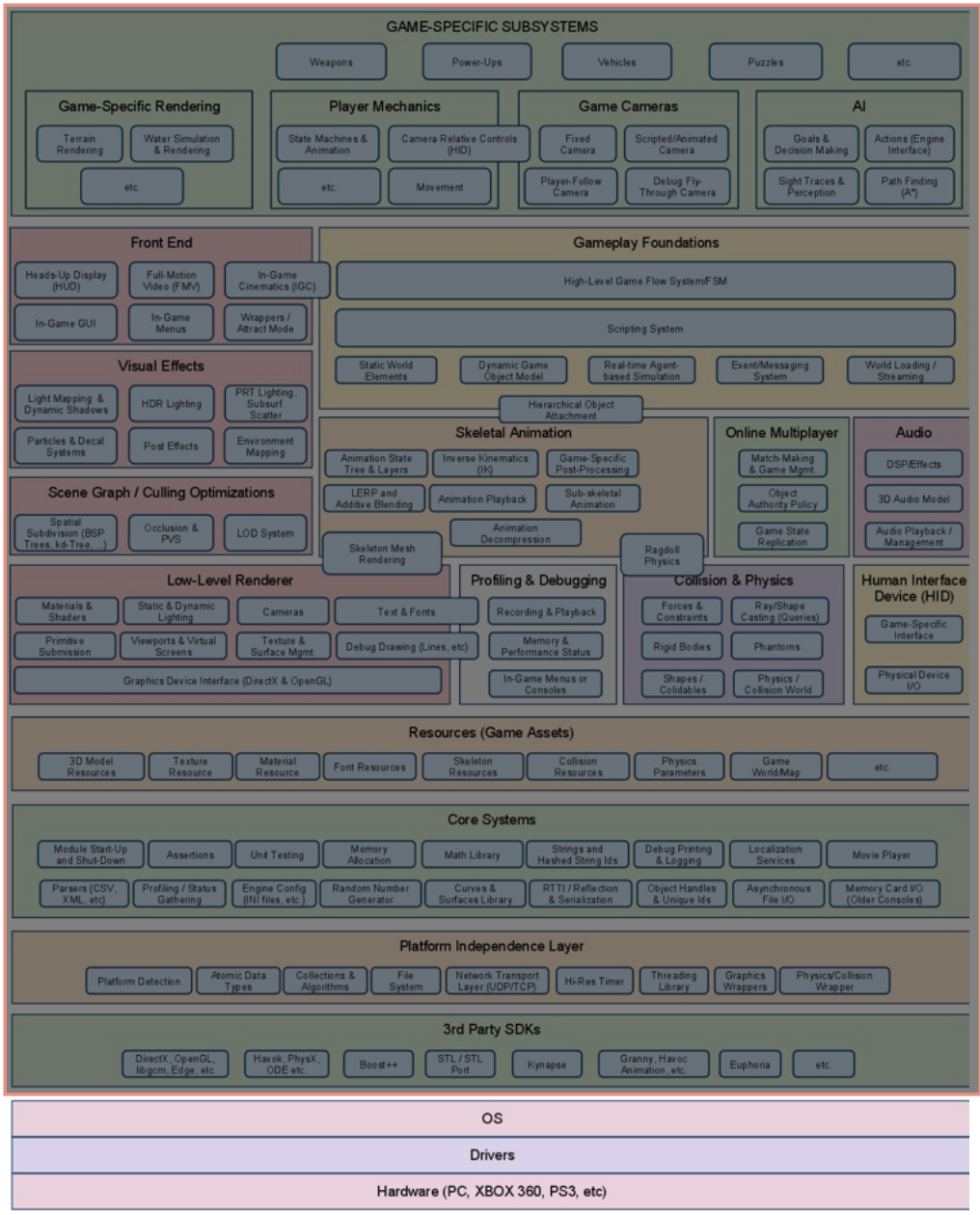
Increasing abstraction

hardware and OS



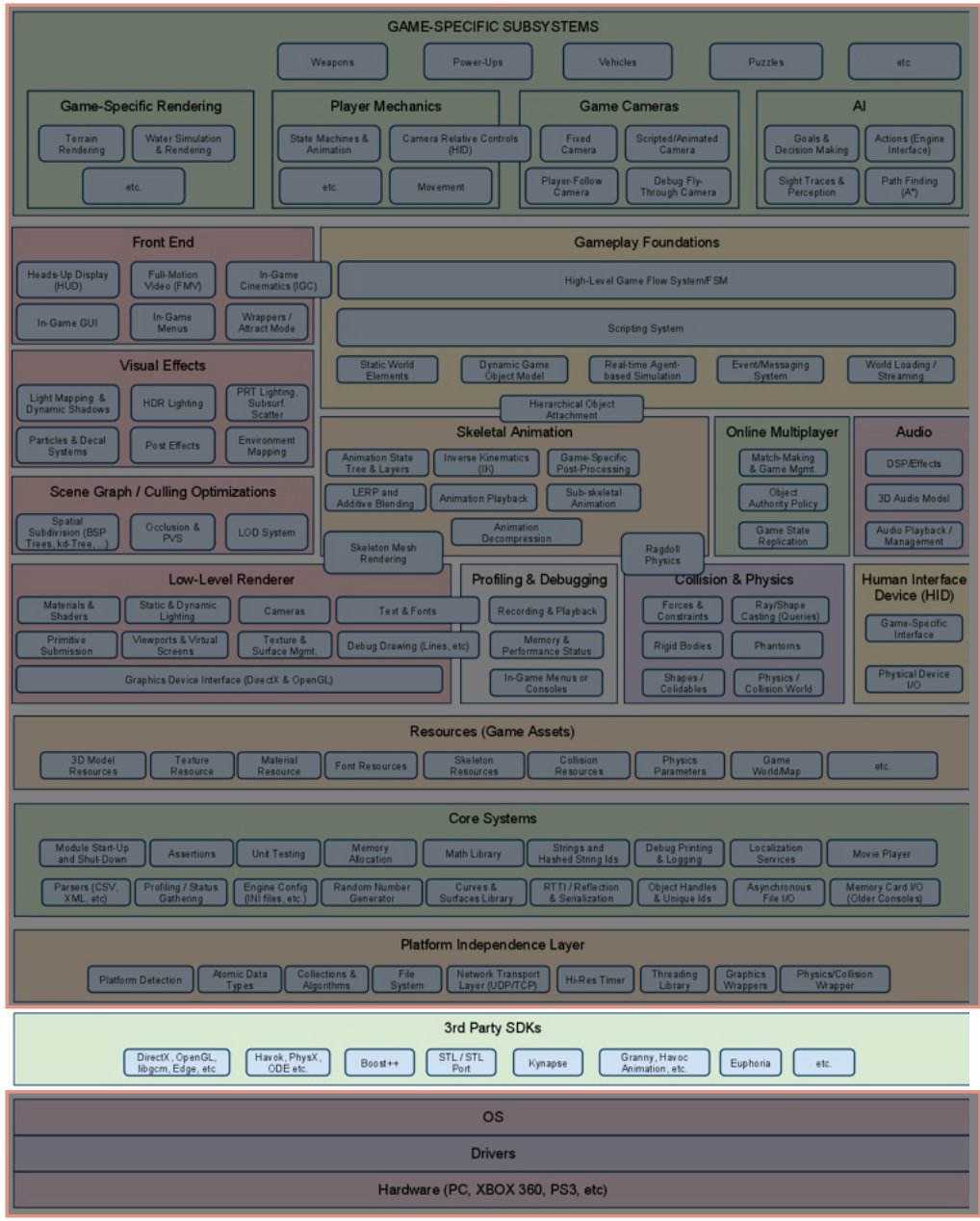
Hardware and OS

- The OS includes the drivers to access the specific hardware devices available.
- In consoles, the OS layer used to be a thin library linked with the game app.
- From Xbox and PS3 onwards, the OS is able to interrupt the game and display notification messages, reducing the gap between computers and console OS.



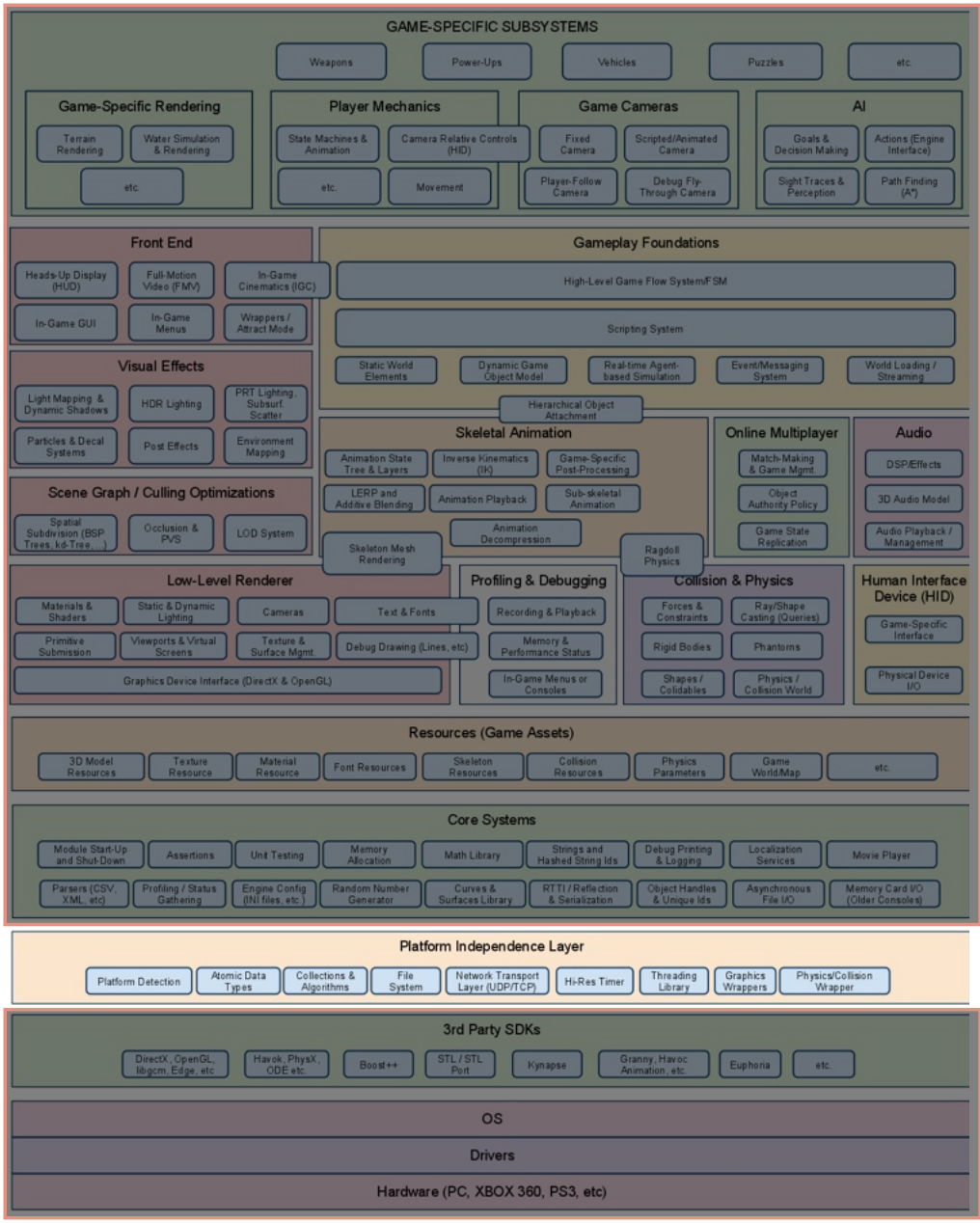
3rd Party SDK and middleware

- Graphics API (DirectX, OpenGL, libgcm, …)
- Collision and Physics (Havok, PhysX, ODE, …)
- Data structures and algorithms libraries (Boost, STL)
- Character Animation support (Granny 3D, Havoc Animation, …)
- AI middleware (Kynapse, …)
- Dynamic motion synthesis based on Biomechanics (Euphoria)
- …



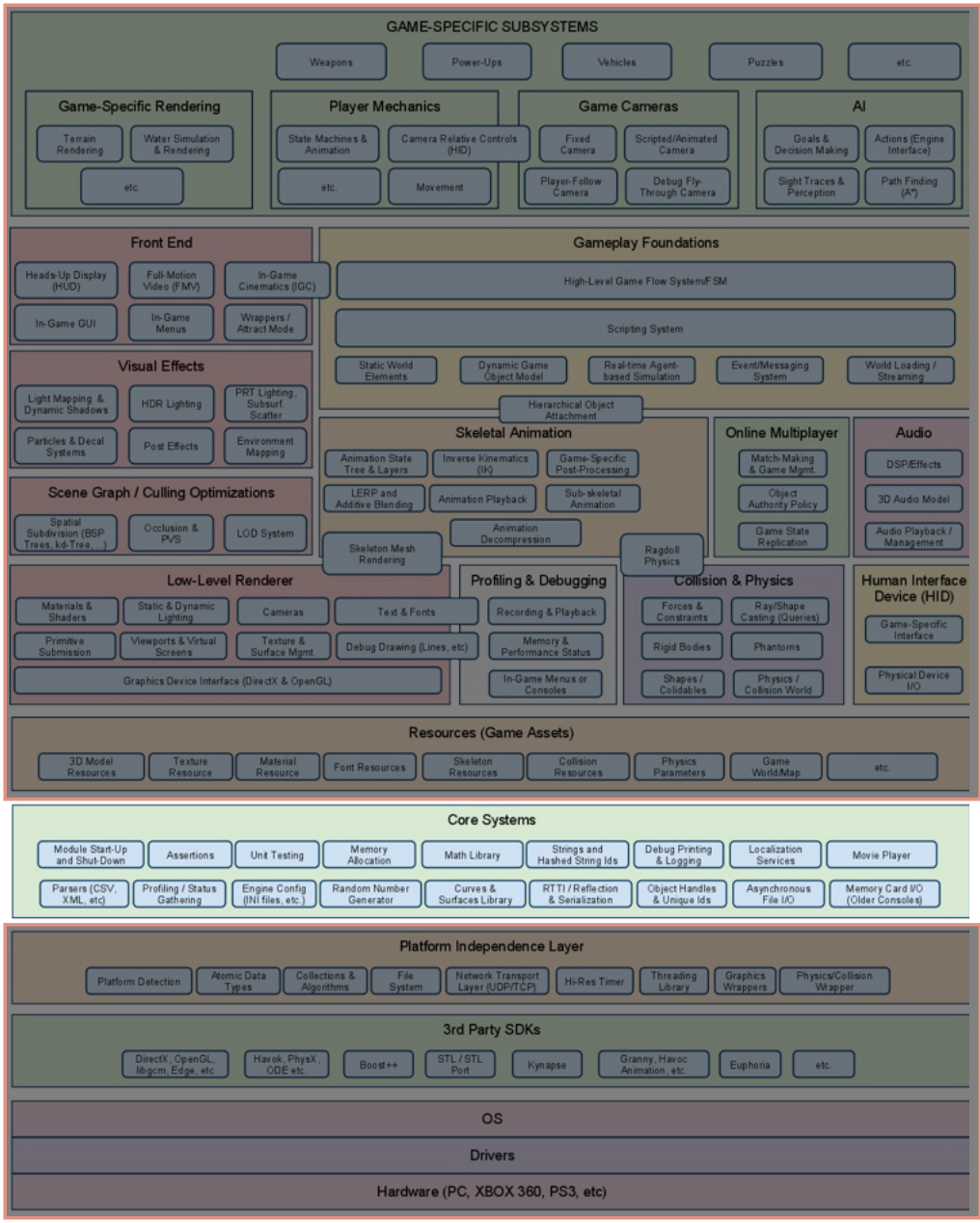
Platform Independence Layer

- Required for all game engines running on different hardware platforms.
- Most studios sell their games for several platforms (except first party studios)
- Shields the rest (top part) of the engine from knowledge of the underlying platform.



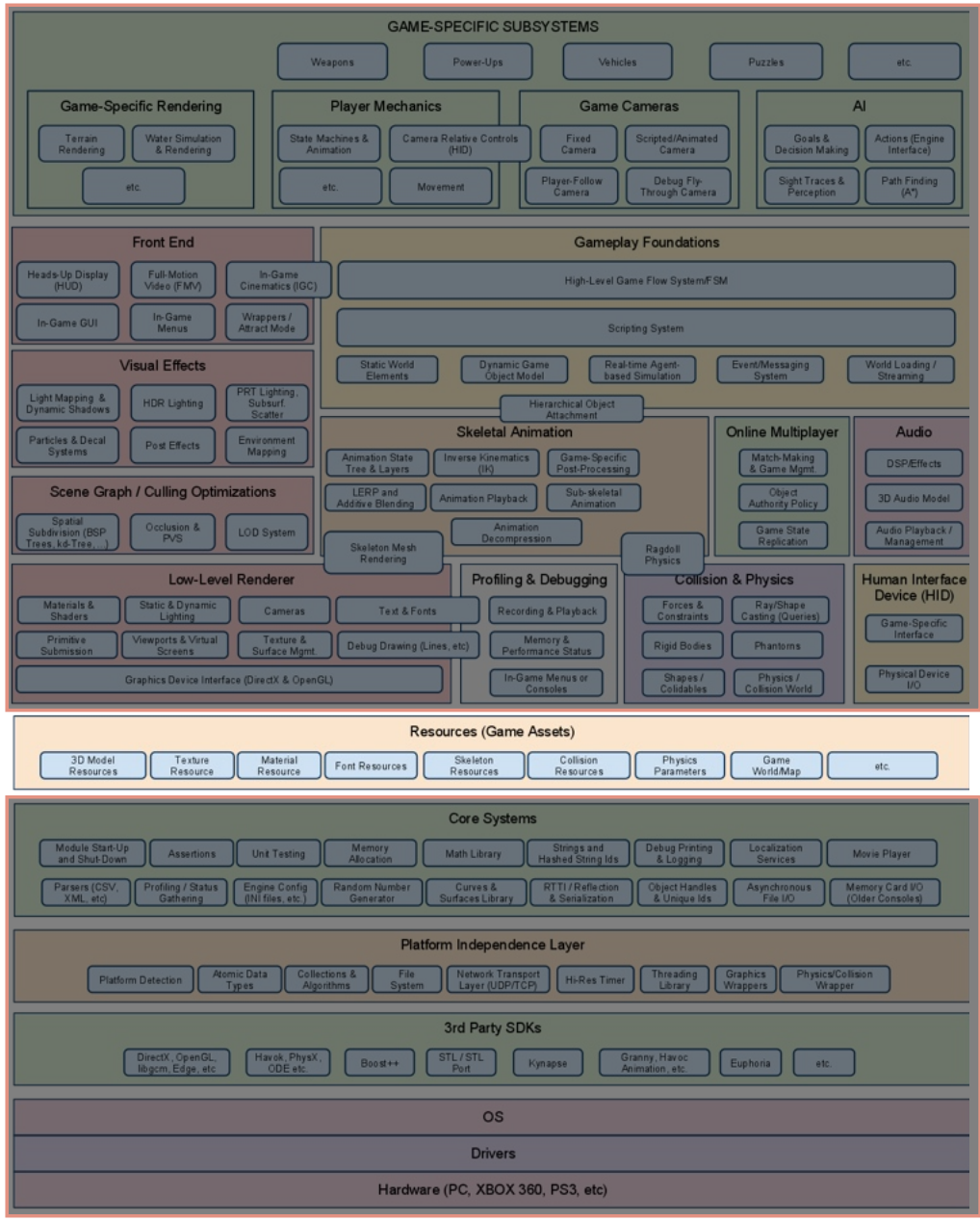
Core Systems

- Examples of “stuff” that go into Core System:
  - Assertions
  - Math library
  - Memory Management
  - Custom Data Structures and Algorithms



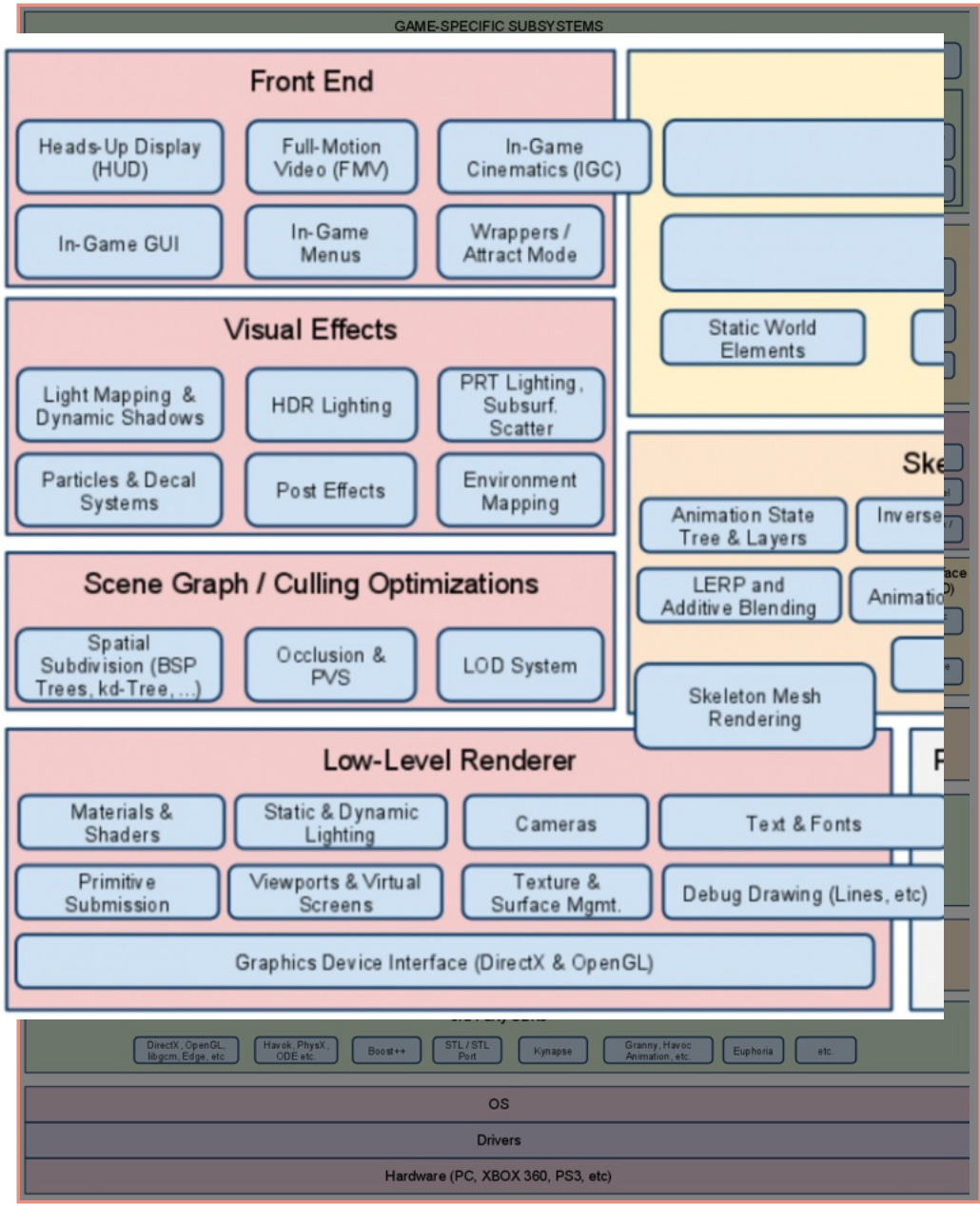
Resource Manager

- Provide interfaces to access game assets and other input data:
  - 3D models, textures, fonts, audio files, skeletons, terrains, …
  - Ad-hoc access to single files vs. resource bundles



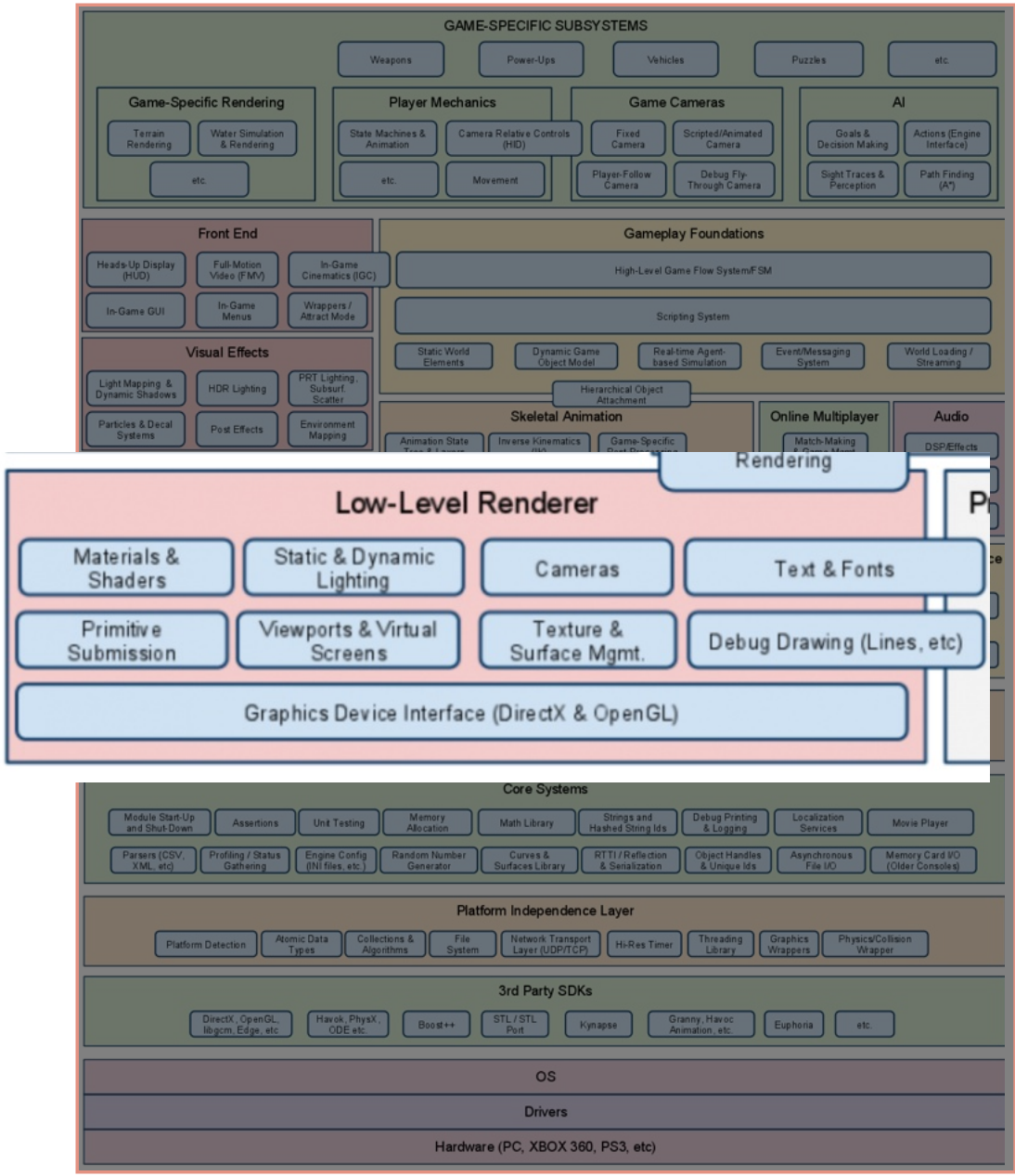
Rendering Engine

- One of the largest and most complex components
- Usually structured in several layers with upper levels providing higher abstraction features



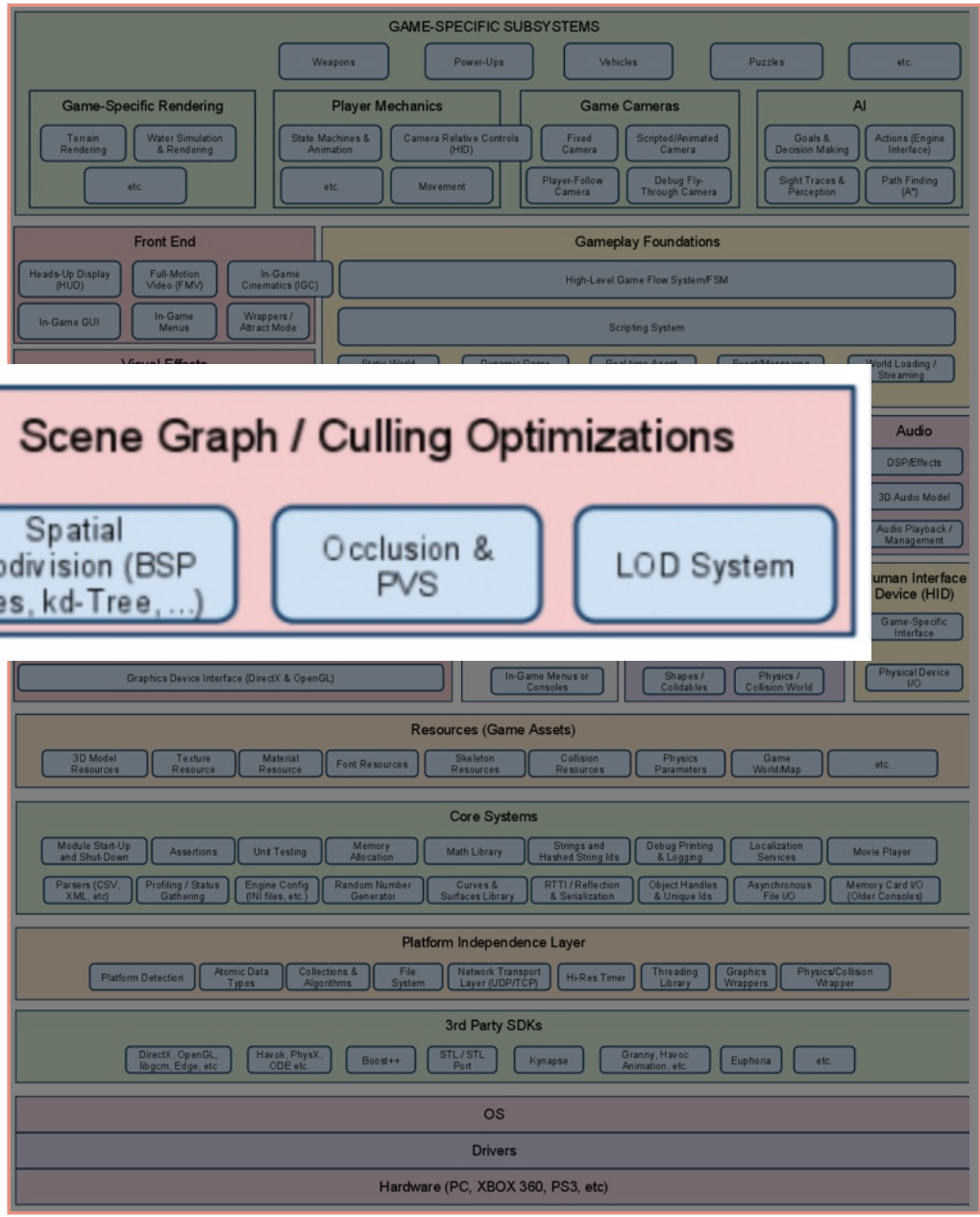
Low Level Rendering

- The lower part deals with graphics API: enumerate the devices, buffer allocations (color, depth, stencil, etc.)
- The upper parts offer support commonly found in 3D graphics pipelines: primitive submission, materials and shader support, textures, lighting, viewports, cameras, …



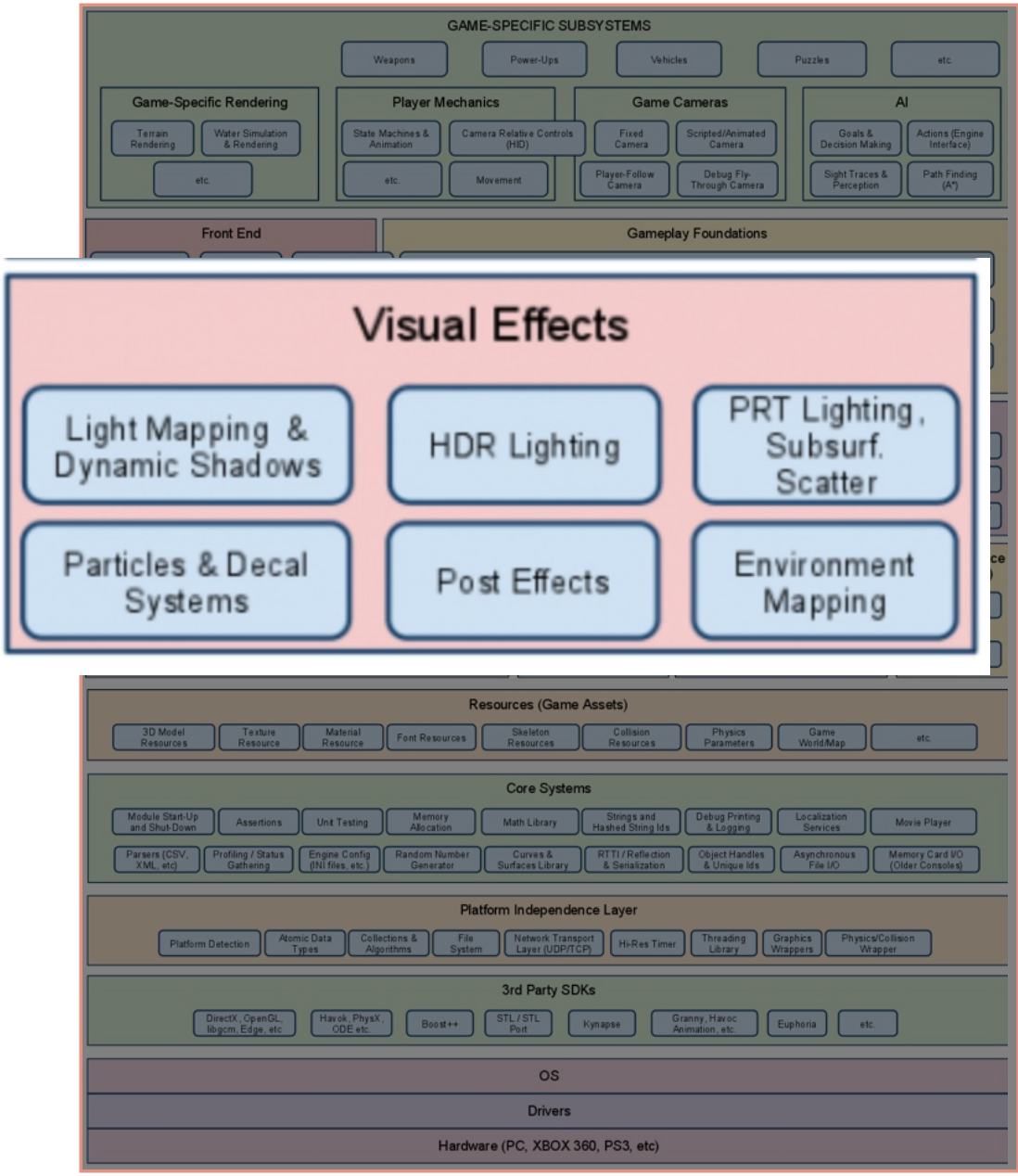
Scene Graph Handling

- Game action can potentially take place in large virtual worlds and/or include a very large number of actors/entities.
- Efficient handling of this large amount of model data is needed so not to stall the pipeline.
- Spatial data structures are employed to discard objects outside the view volume.
- Objects may occlude other objects and lead to further optimizations.
- Objects at different distances from the camera may be rendered with different levels of detail.



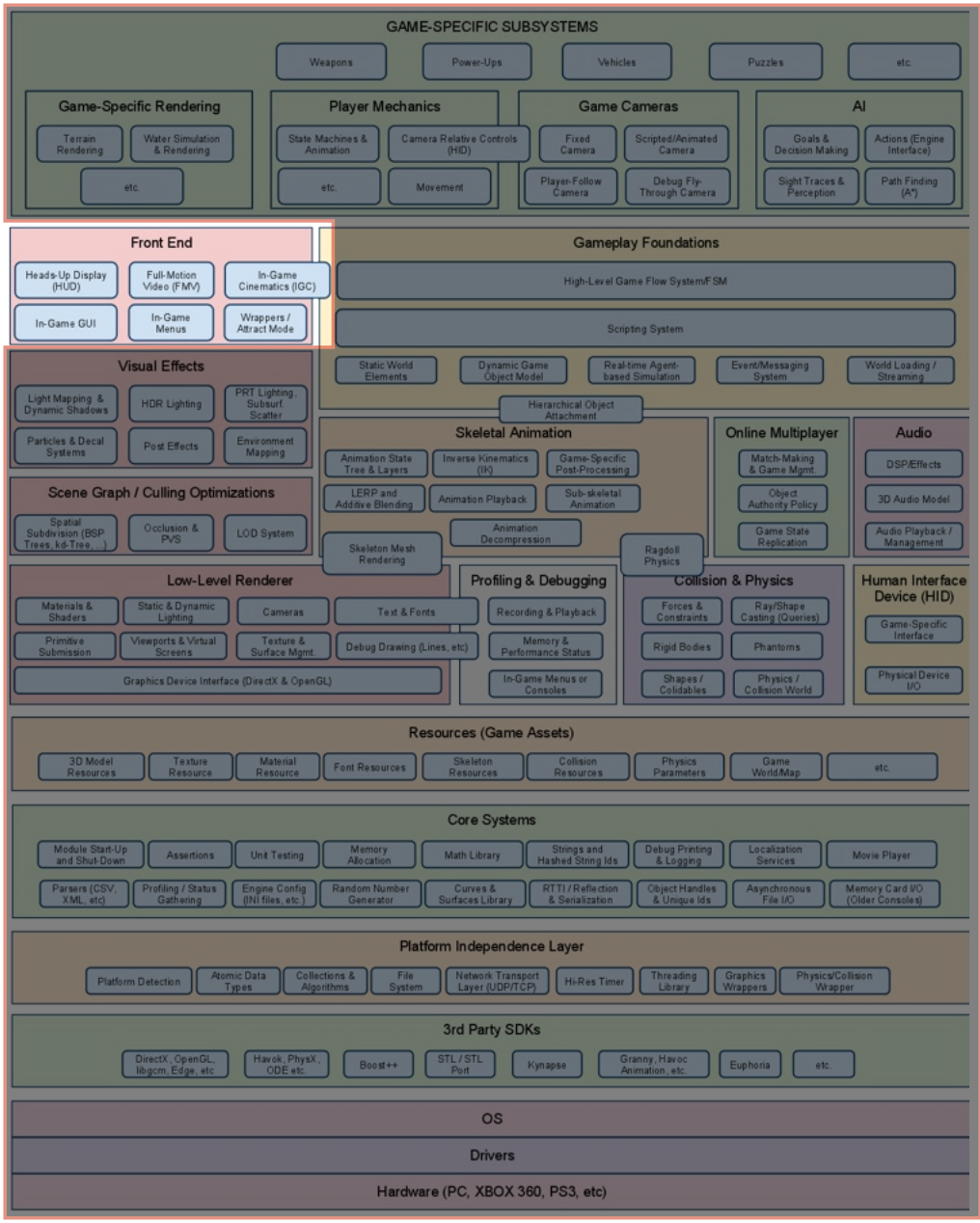
Visual Effects

- Modern video games include highly advanced visual effects:
  - particle systems (water splashes, smoke, fire,…)
  - decals (bullet holes, footprints, …)
  - light mapping and environment mapping
  - dynamic shadows
  - full-screen post effects (HDR, FSAA, color correction/shift)



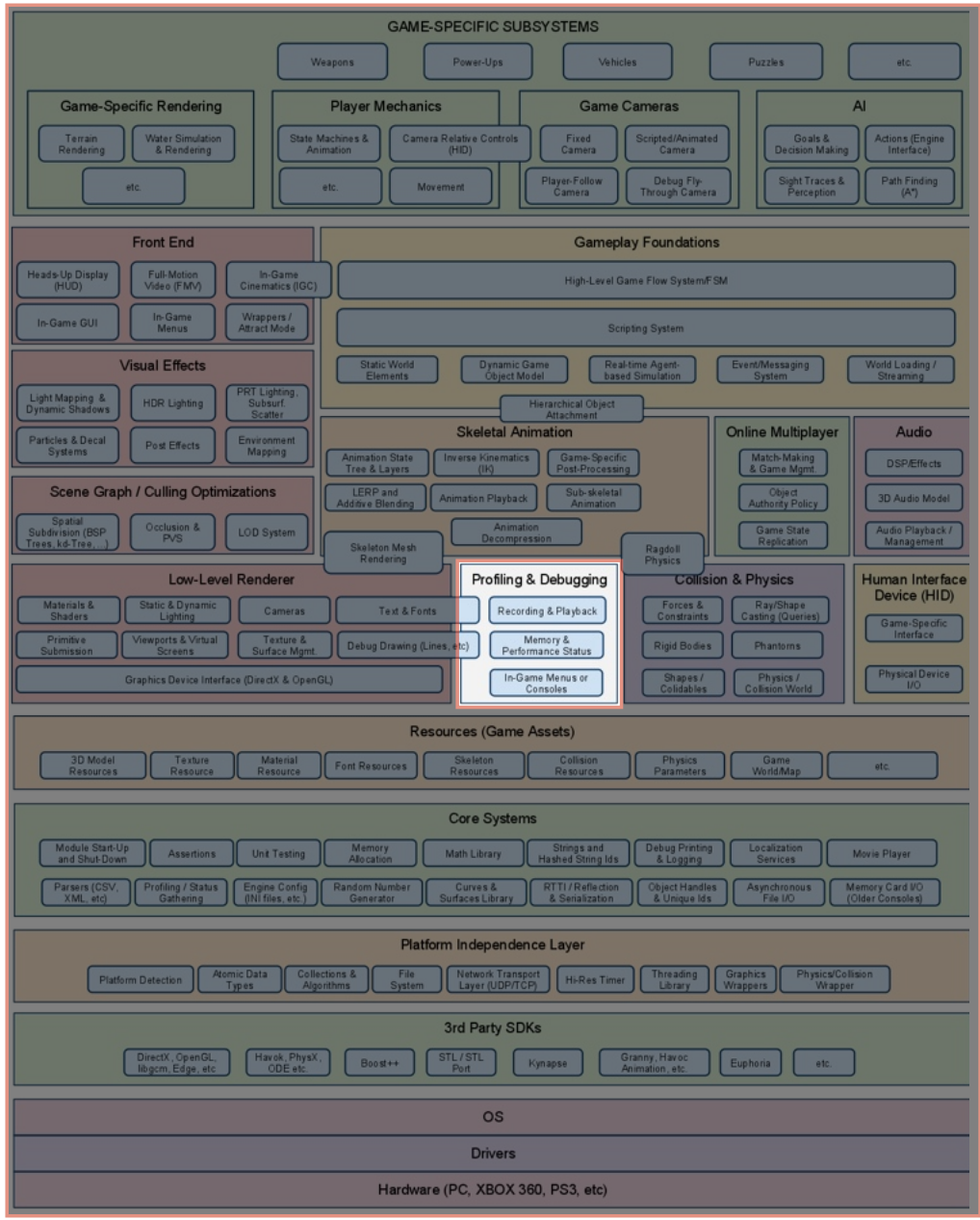
Front End

- Examples:
  - Heads-up display
  - In-game menus, console, development tools (may not ship with the game)
  - In-game GUI to manipulate the character's inventory, deploy units to battle, …
  - Video playback and scripted Cinematics Choreography using the Engine
- A large portion of these features require making 2D textures to the screen (orthographic view) or 3D billboards (always facing the camera).



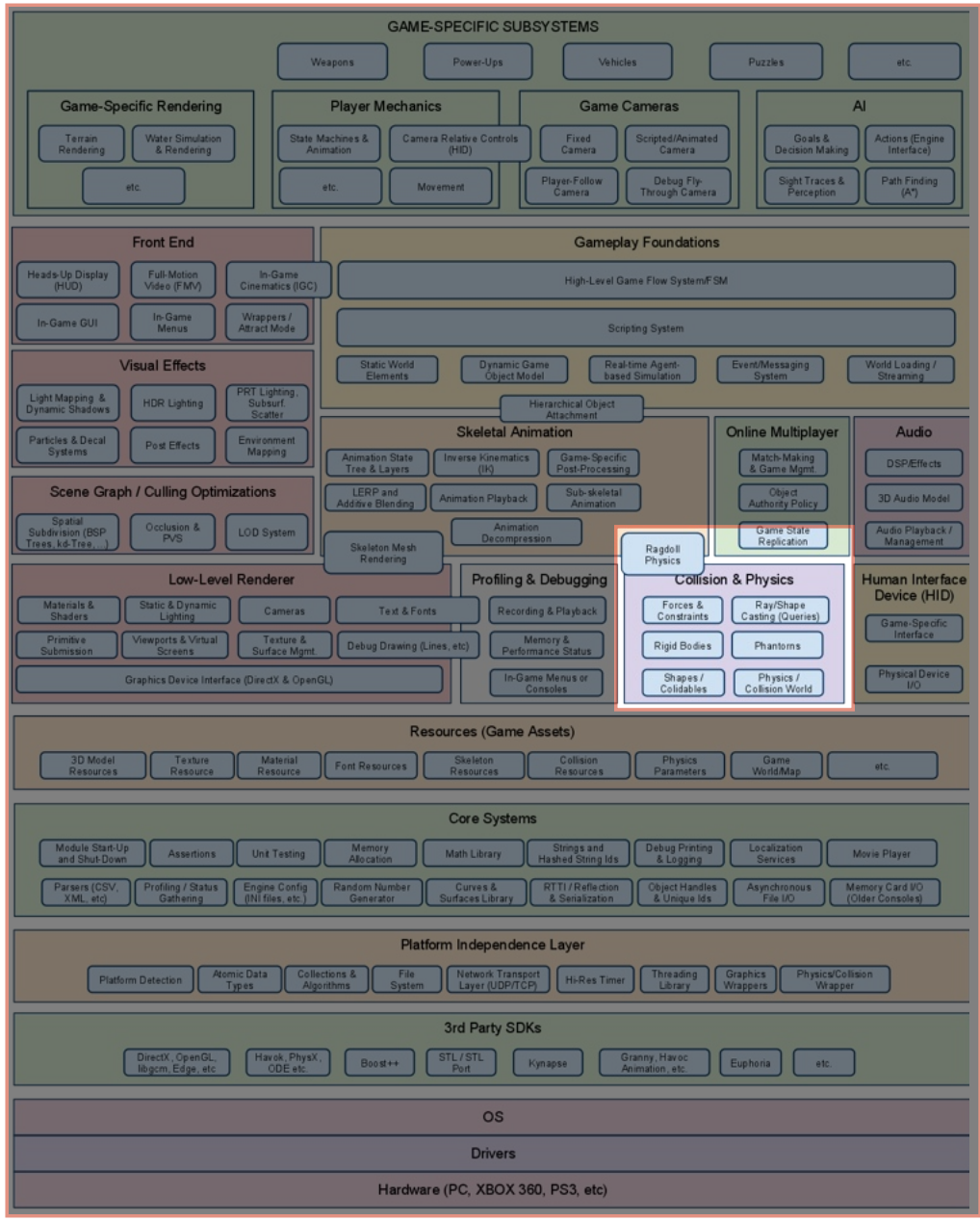
Profiling and Debugging

- Performance is a top priority in game development
- Console hardware quickly becomes outdated and resources are scarce.
- In-game debugging tools often include a development console and drawing facilities.
- Recording, playback, stopping the game clock, dumping stats, etc…



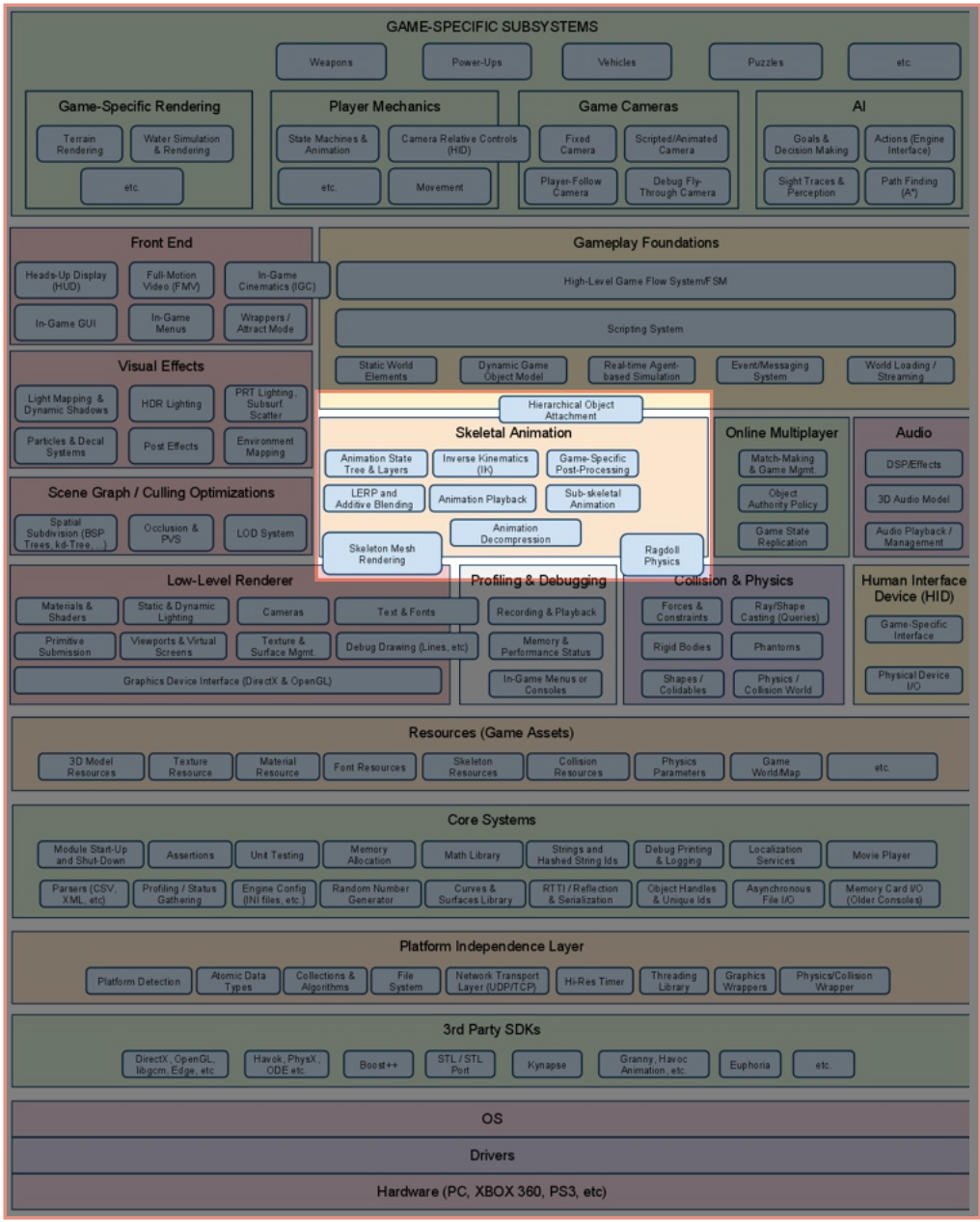
Collisions and Physics

- Collisions is at the heart of every game. Without them, objects would overlap and no interaction with the virtual world would be possible.
- Collision is commonly done with low detail, simpler, shapes
- The Physics system comprises rigid body dynamic simulation based on forces, torques and restrictions.



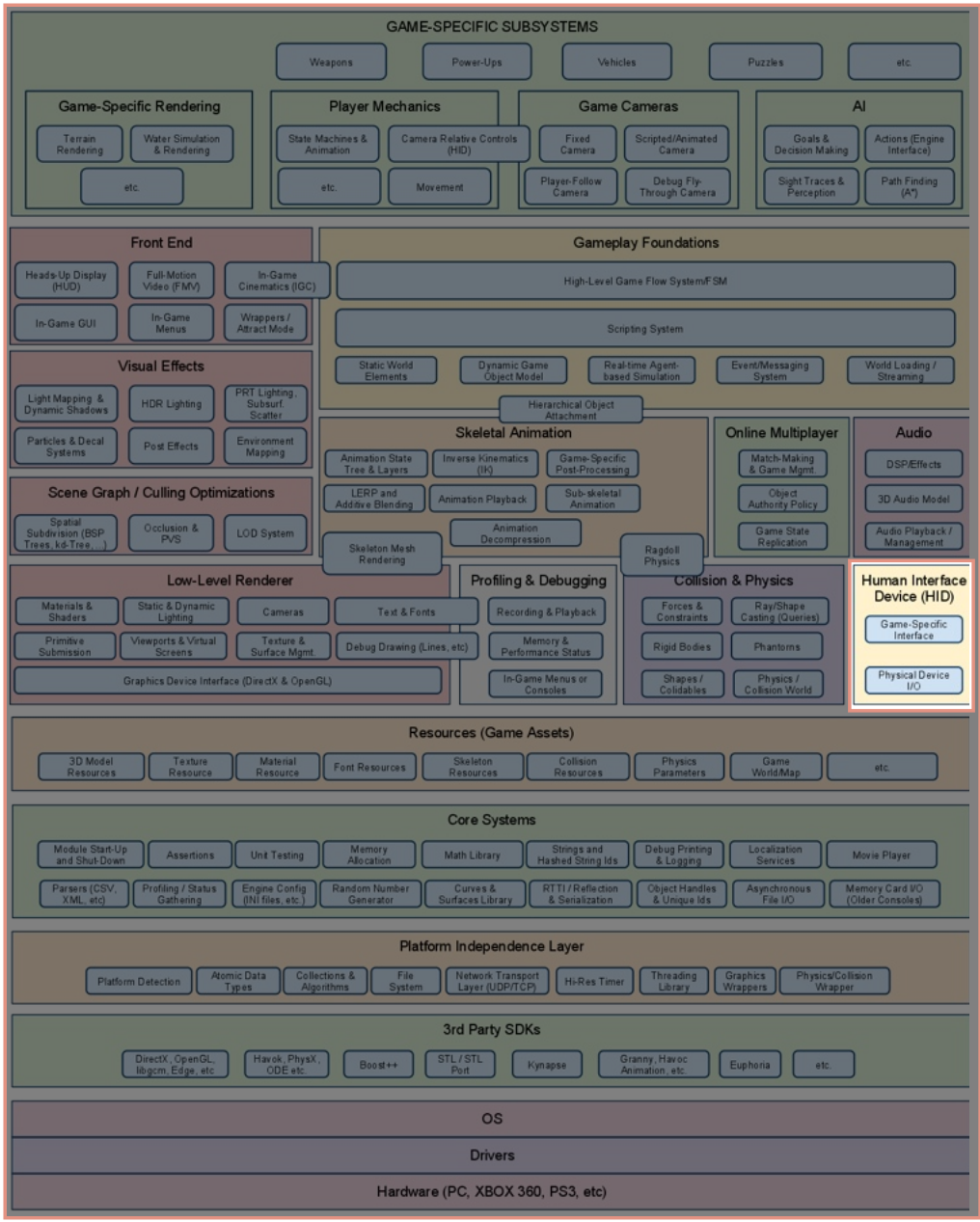
Animation

- Animation systems are needed for modeling humans, animals, cartoon characters, robots,…
- types of animation systems:
  - sprite/texture animation (2D)
  - rigid body hierarchy animation (machines, robots)
  - skeletal animation (humans, animals, cartoon characters,…)
  - vertex animation
  - morph targets



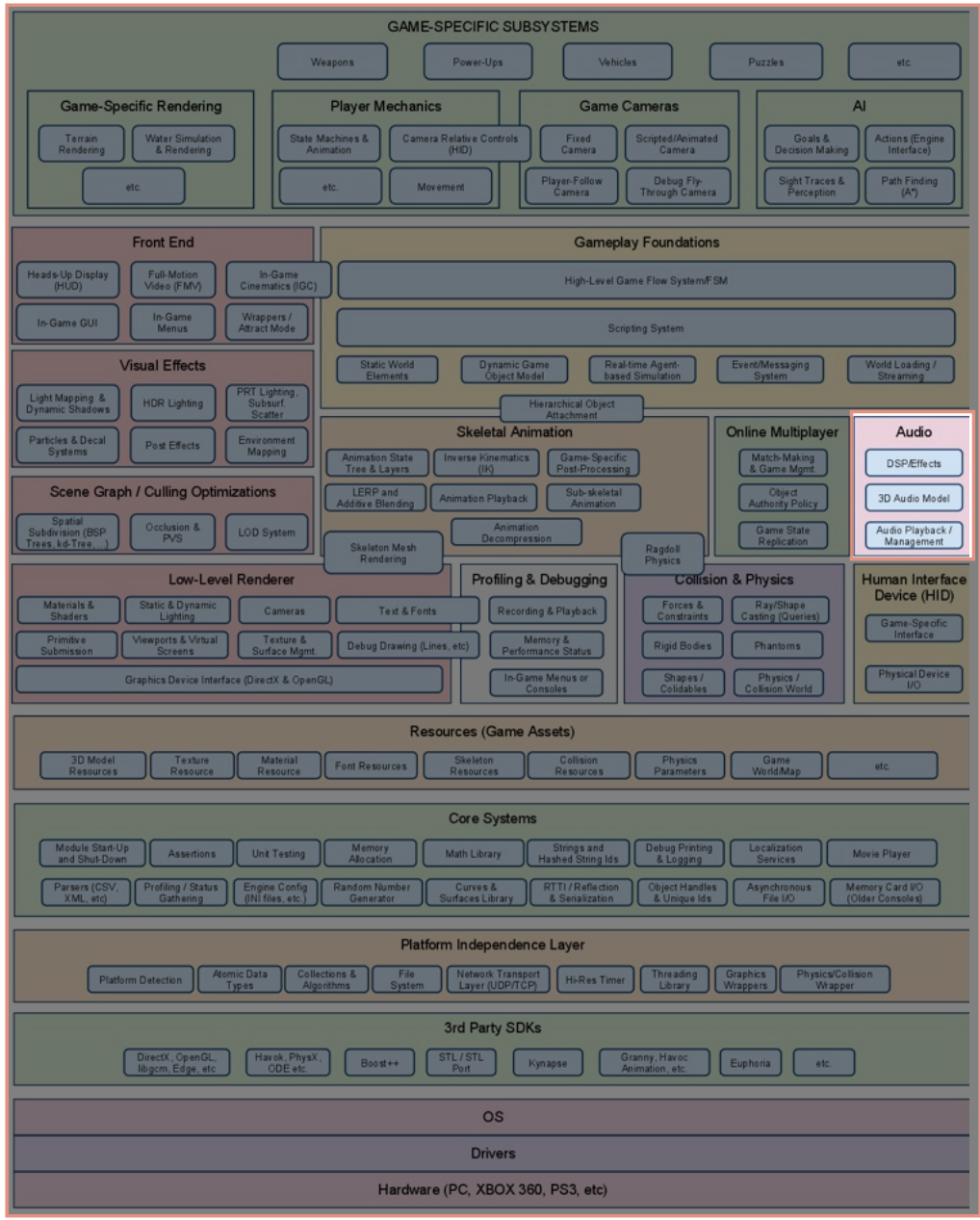
Human Interface Devices

- Process user input from HID:
  - keyboard, mouse, joypad, specialized game controllers
- process raw data:
  - dead zone around center position of joypad stick
  - debounce button presses
  - interpret and smooth signals from accelerometer
  - allow for user customized mappings
  - detect chords, sequences and gestures



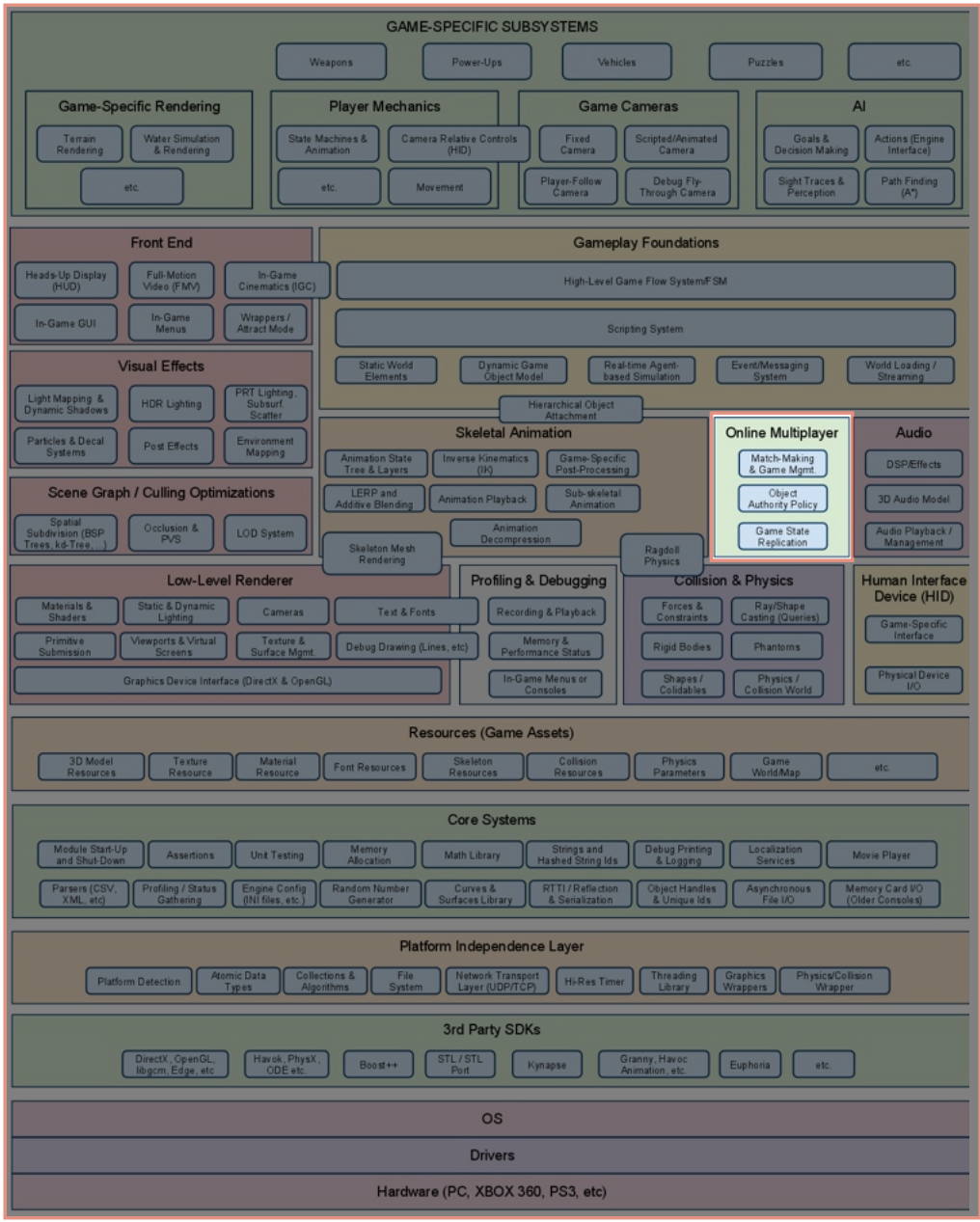
Audio

- Just as important as rendering and physics
- Essential to create immersive experiences
- Can be as simple as basic playback but often includes 3D spatialization and DSP/Effects
- Examples of Audio engines: X3DAudio, SoundR!ot, Scream



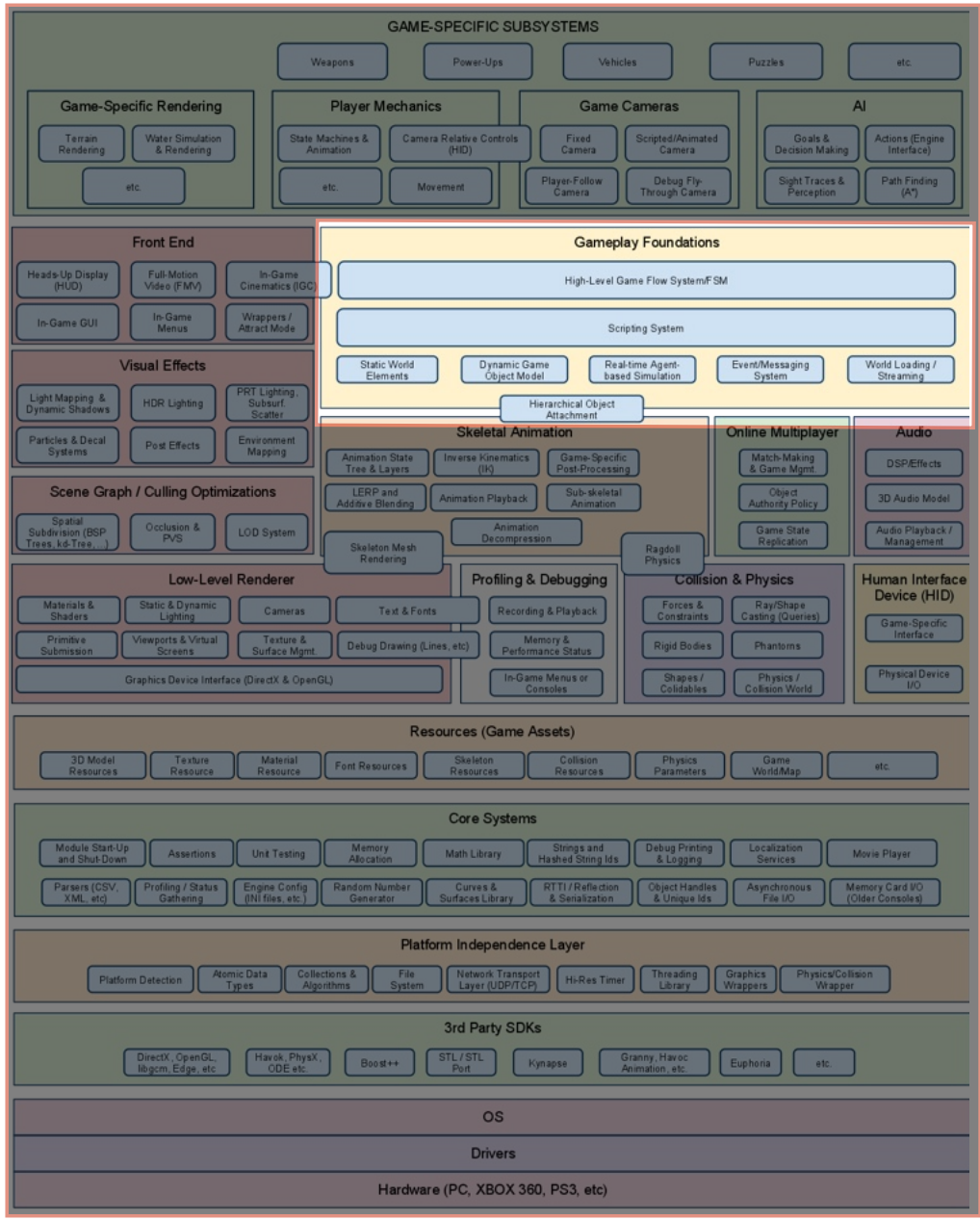
Online Multiplayer/ Networking

- Allow for multiple human players in the same virtual world
- 4 flavor:
  - Single-screen multiplayer
  - Split-screen multiplayer
  - Networked multiplayer - Each machine hosts one player
  - Massively multiplayer online games (MMOG) - A battery of central servers host a common and persistent game world.
- Easy to change a multiplayer game into a single player, harder the other way around.



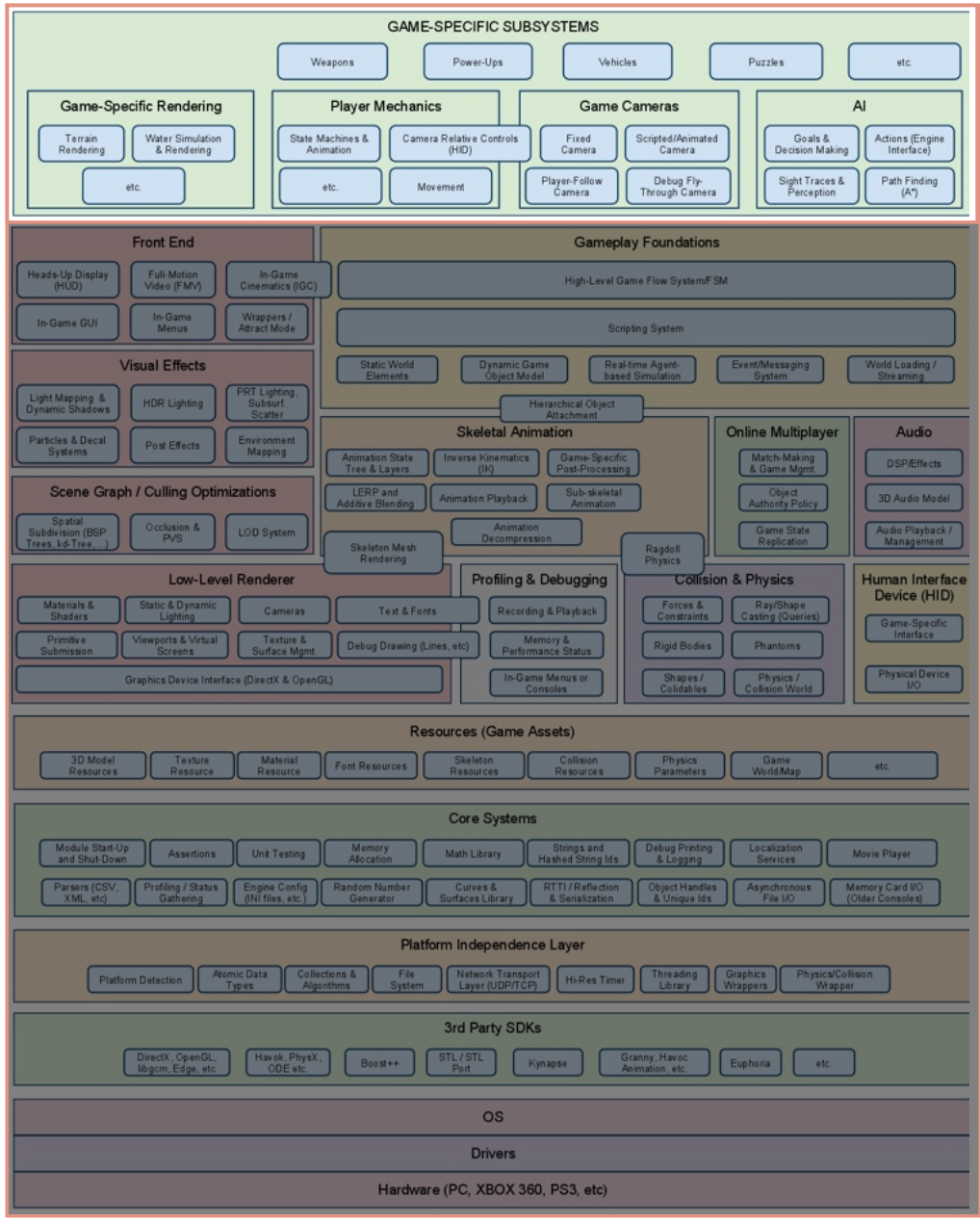
Gameplay Foundation Systems

- Includes:
  - Game world and object models for static background, dynamic rigid bodies, PC and NPC, weapons, projectiles, vehicles, lights, cameras,…
  - Event system for inter object communication
  - Scripting system for easier and more rapid development of game rules
  - AI foundations



Game Specific Subsystems

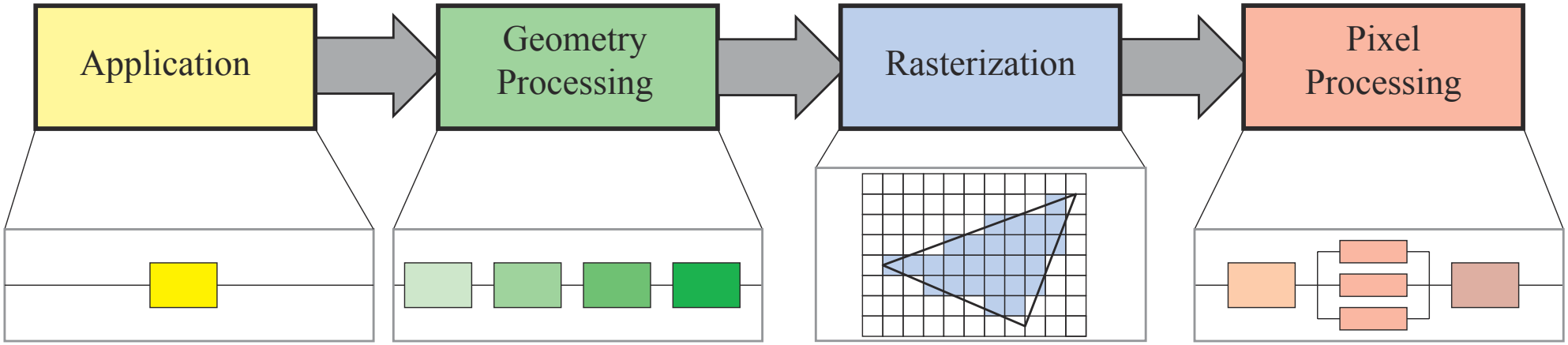
- Everything that is required by the game but not included in the game engine…
- Our contributions to our own games form this block



Graphics Rendering Pipeline

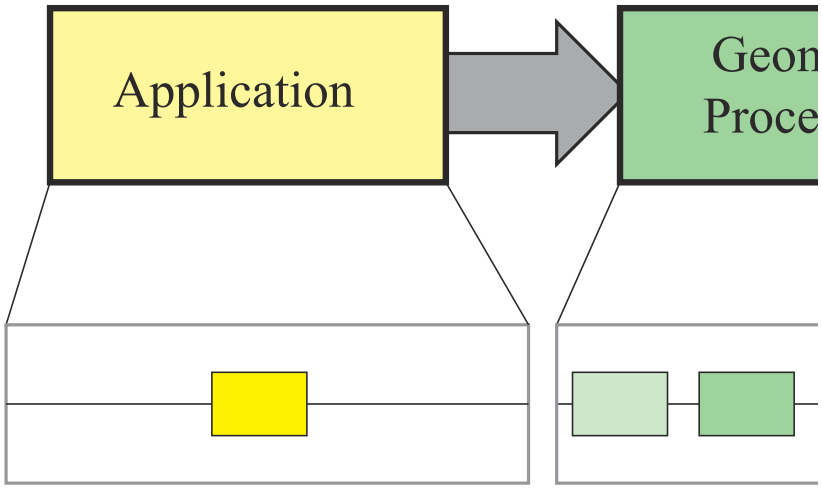
Fernando Birra
Rui Nóbrega

Architecture



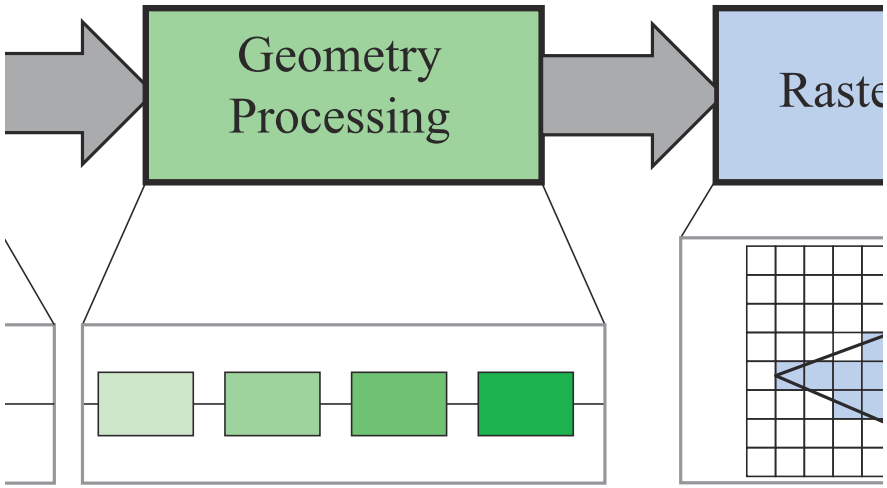
Application

- Driven by the application and is typically implemented in software, on the CPU
- Several cores can be used to handle different tasks in parallel such as collision detection, acceleration algorithms, animation, physics simulation, handle input, …
- The developer has full control of this stage
- What is done here can affect the performance of later stages. For instance, a better geometry pruning algorithm may significantly reduce the number of primitives to be rendered
- The GPU can help in this stage by using “compute shaders”
- The final goal of this stage is to feed geometry data to the next stage (rendering primitives)

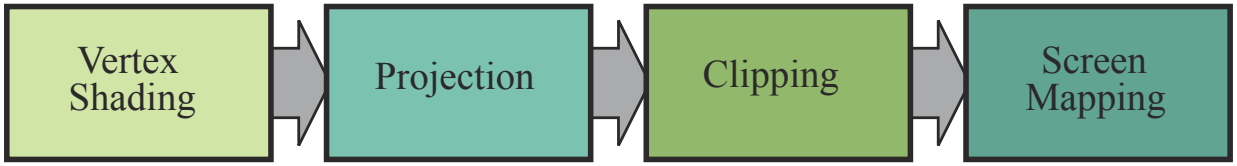


Geometry Processing

- Responsible for applying transformations, perform projection and additional handling of geometry
- Determines what should be drawn, how it should be drawn and where it should be drawn
- It is generally executed in the GPU using many cores and fixed-operation hardware

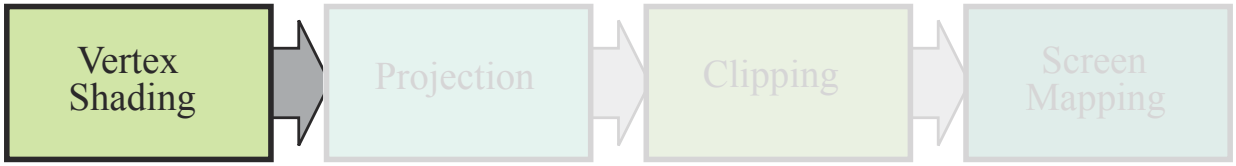


Geometry Processing



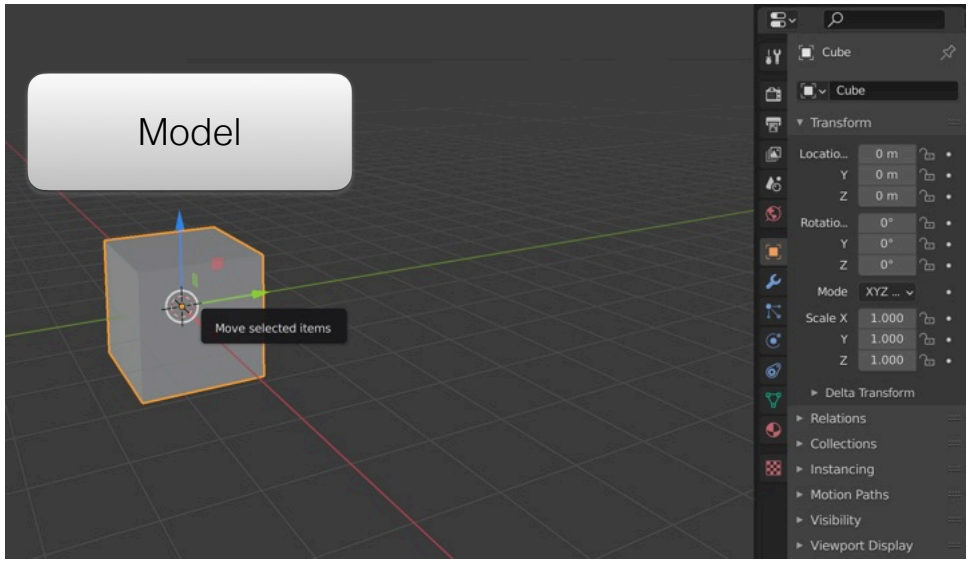
- It is subdivided in 4 functional stages

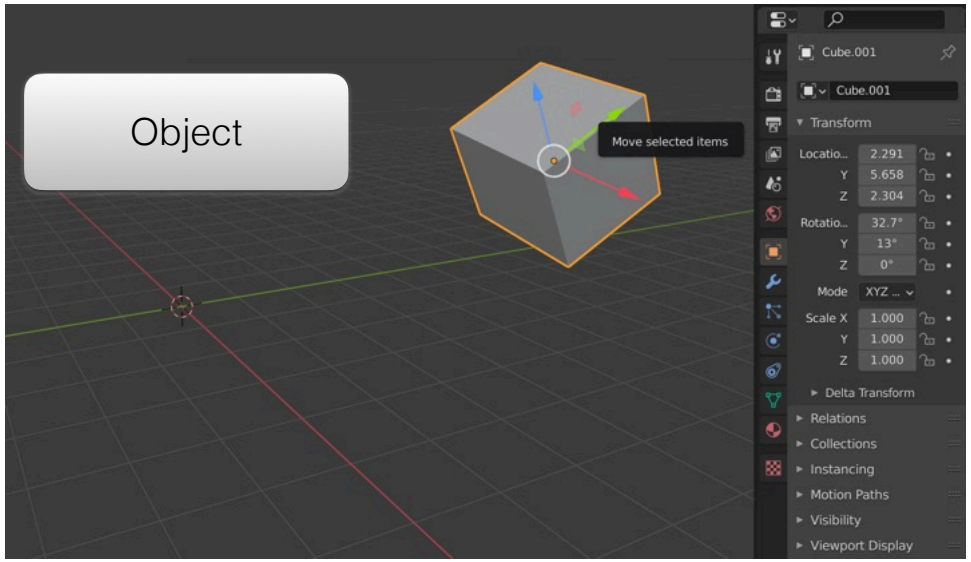
Geometry Processing - Vertex Shading



- Major tasks to be performed:
  1. compute the position for a vertex
  2. specify the outputs per vertex (normals, texture coordinates, …)
- The “vertex shader” name comes from the fact that, in the past, it was common to compute the final color for each vertex and have those colors interpolated across a triangle.
- More powerful hardware allowed the migration of the final color computation to be performed on a pixel level…
- … the vertex shader is now a general unit dedicated to setting up the data for each vertex (interpolation, animation, …)

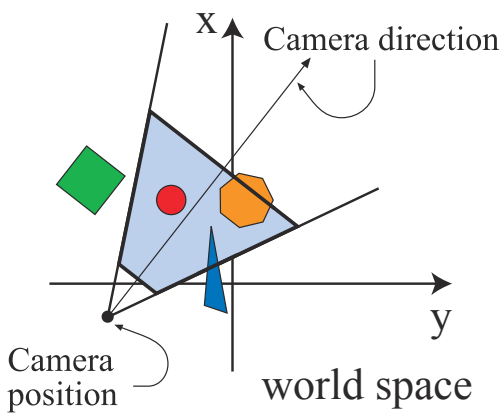
1. Compute the position of a vertex





- Go from models to objects in the scene, as seen by the camera, by applying a modeling transformation to both vertices and normals
- The same model (base geometry) can be transformed using different modeling transformations (one for each instance)
- Original coordinates are defined in **local coordinate system**, intermediate coordinates are in a **global/common coordinate system** (World Coordinates), final coordinates are in **camera coordinate system**.

1. Compute the position of a vertex

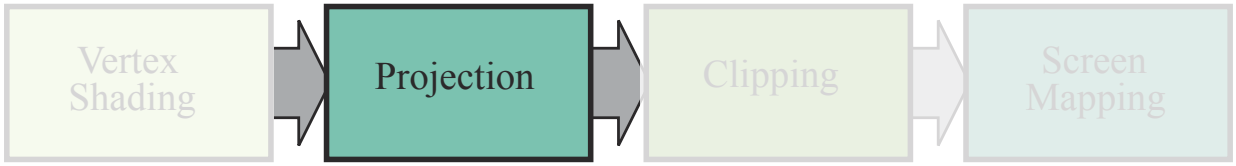


- Only objects captured by the camera need to be rendered
- The camera has a location and orientation in the world and it can be used to transform the vertices of the instances from world to camera coordinates (view transform)
- The local camera coordinates make projection and visible surface determination easier
- In camera coordinates, the camera is at the origin, looking towards the negative* z axis, with y pointing upwards
- Both the **model** transform and the **view** transform can be implemented using 4x4 matrices and even be combined in a single matrix

2. Specify the outputs per vertex

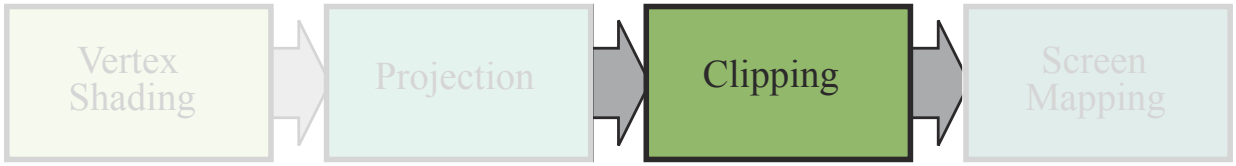
- To generate a realistic scene we need more than geometry…
- The appearance of the objects must be modeled using materials and light sources
- Determining the effect of light on a surface is called shading and it involves evaluating a shading equation at several points of an object.
- From a set of per vertex input data (such as location, normals, texture coordinates, material properties or color) vertex shading results (which can be colors, vectors, texture coordinates, …) are sent to the rasterization and pixel processing stages to be interpolated and used in the final shading of the surface.

Geometry Processing - Projection



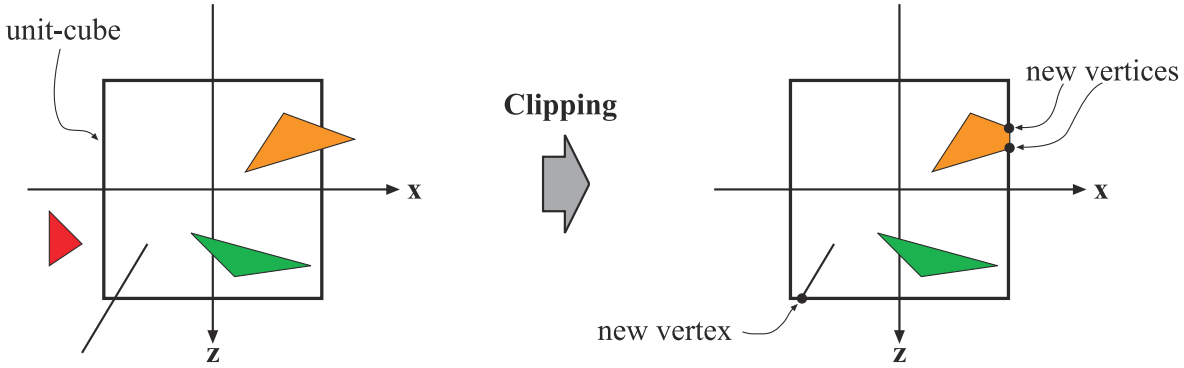
- Projection is the transformation of the view volume into a canonical view volume (typically a cube ranging from (-1,-1,-1) to (1,1,1)
- Both parallel and perspective view volumes (rectangular box and a truncated pyramid with rectangular base, respectively) can be transformed to this canonical view volume, simplifying the operations performed next
- Projection can also be expressed as a 4x4 matrix and concatenated with the previous transformations
- Although these transformations change a volume into another, they are called projections because after displaying the image, the z coordinate is stored in a different place (z-buffer) and the image generated is thus 2D only

Geometry Processing - Clipping



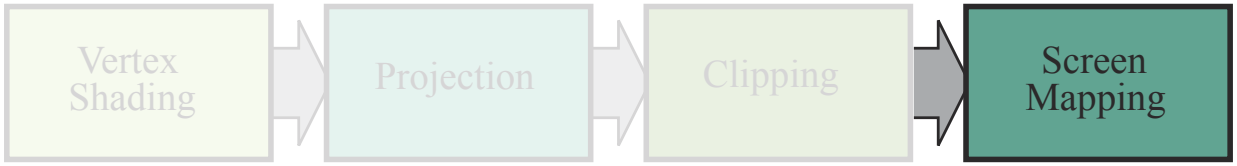
- After the projection transformation the objects are said to be in clip coordinates
- The GPU vertex shader must always output the corresponding clip coordinates of the input vertex
- Clipping is the process of throwing away the parts of a primitive that do not lie inside the canonical view volume.

Geometry Processing - Clipping



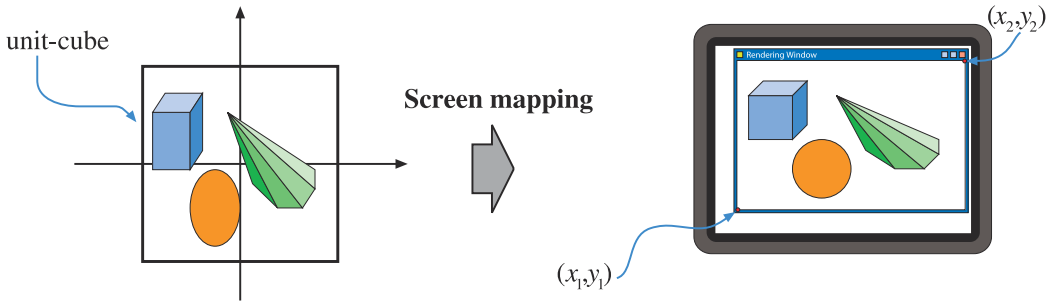
- After the projection transformation the objects are said to be in clip coordinates
- The GPU vertex shader must always output the corresponding clip coordinates of the input vertex
- Clipping is the process of throwing away the parts of a primitive that do not lie inside the canonical view volume
- Clipping is performed in homogeneous coordinates

Geometry Processing - Screen Mapping



- In the final stage, after clipping, the projected objects are transformed to screen space
- The rectangular area spanning from (-1,-1) to (1,1) is transformed to screen space using a viewport defined in integer screen coordinates.
- Screen coordinates together with the z coordinate are also called window coordinates.

Geometry Processing - Screen Mapping

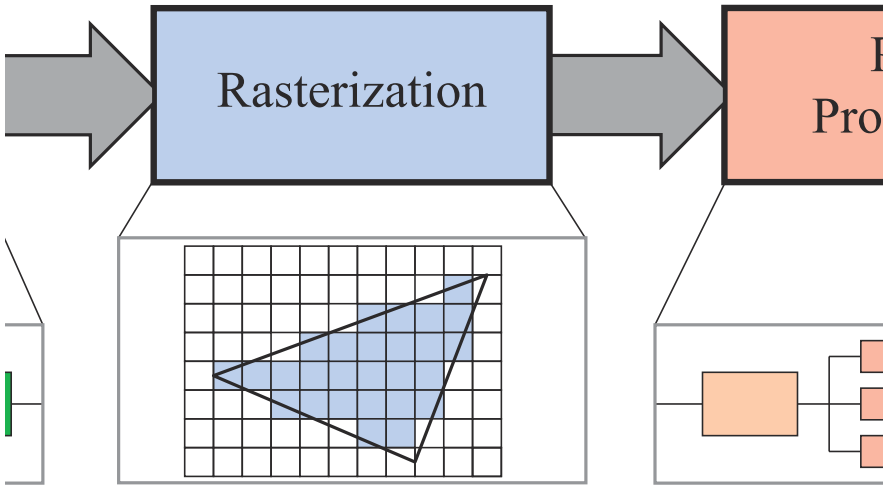


- The viewport spans from (x1, y1) to (x2, y2)
- Pixel centers have their locations at .5 (0.5, 1.5, 2.5, …)
- Left/bottom* side of pixel 0 has x/y coordinate equal to 0
- Left/bottom* side of pixel 1 has x/y coordinate equal to 1

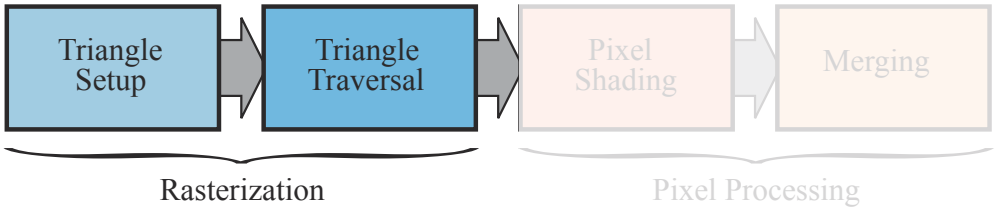
Rasterization

- Takes as input a set of transformed and projected vertices and it finds the pixels that are considered as part of the primitive being drawn

- For a triangle, each set of three vertices result in a set of pixels considered to be inside the triangle (pixel centers inside the triangle) and that need to be further processed



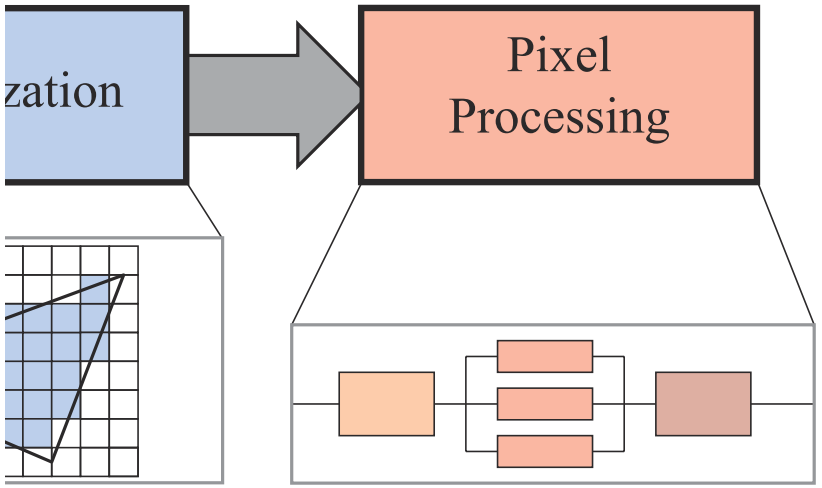
Rasterization



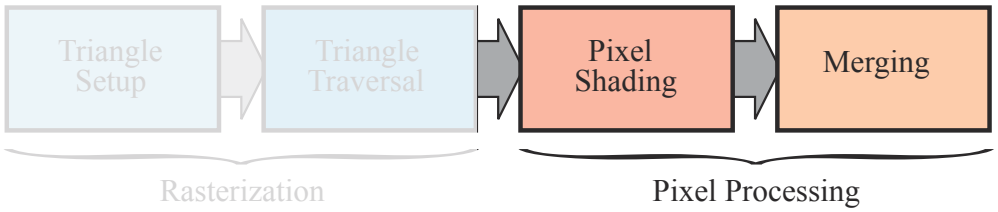
- Rasterization is subdivided in two functional stages
- The first stage is called Triangle Setup (or Primitive Assembly) and it picks groups of vertices to form the primitive (1 for points, 2 for lines and 3 for triangles). Triangles are born here! Differentials and edge equations are computed in this stage
- Triangle traversal generates a fragment for each pixel that has its center point covered by the primitive. Data associated with each fragment is interpolated from the data generated at each vertex of the primitive (includes depth and any shading data that was generated during vertex shading)
- All samples that are inside a primitive are then sent to the pixel processing stage

Pixel Processing

- In the final stage, each pixel is “shaded” by a program that computes its final color and may perform depth testing to determine if the pixel is visible
- Other per pixel operations such as blending the newly computed color with a previous color is also possible
- The rasterization and the pixel processing stages are totally performed in the GPU



Pixel Processing



- This stage is divided in two functional stages
- **Pixel shading** is performed on every sample using interpolated data generated before. This executes on a GPU and it is fully programmable. The pixel shader is provided by the programmer to shade each pixel and texture mapping is usually performed here.
- After a final color is computed for each fragment, it needs to be combined (**merged**) with whatever lies on screen at the output location. It is not a fully programmable stage but a highly configurable one. The visibility test for the fragment is performed in this stage (Z-Buffer)
- Besides color (frame buffer) and depth (z-buffer), we can also use the alpha channel (part of the frame buffer that handles transparency/ opacity), the stencil buffer (a buffer where a primitive can also be written and that can be used to control rendering to the color buffer and the z-buffer)
- The operations at the end of the pipeline are raster operations or blend operations. The frame buffer is normally used to refer to all the buffers in the system. Double and triple buffering (two/three color buffers) is used to avoid flickering or image tearing.

Pipeline Optimization

Fernando Birra
Rui Nóbrega

Pipeline Optimization

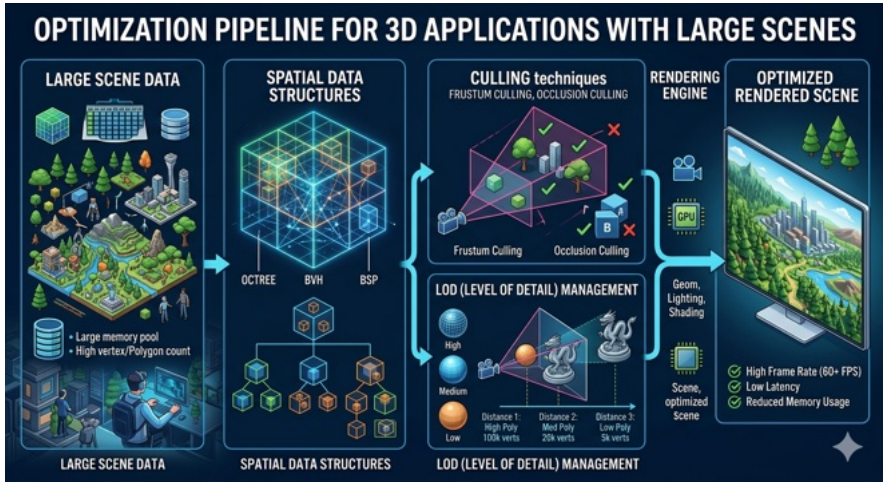
- While it is common sense that we can impose limits on (due to diminishing returns):
  - Resolution
  - Framerate
  - Shading complexity
- There is no upper limit on scene complexity!
- Need ways to deal with large scene graphs and massive detail models

Pipeline Optimization

- The strategy is applied in stages, by applying acceleration algorithms:

  1. Use culling techniques to eliminate non visible or negligible parts

  2. Use level of detail techniques to reduce complexity of remaining objects



Spatial Data Structures

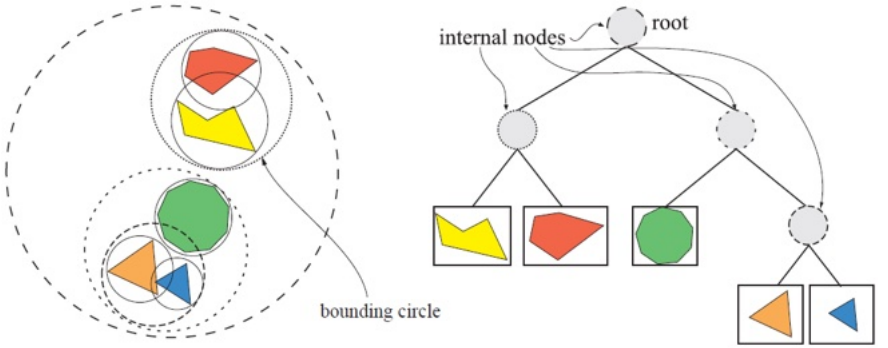
- Many acceleration algorithms heavily rely on using spatial data structures
- A spatial data structure organizes geometry in some n-dimensional space (most commonly in 2D or 3D)
- They can be used to accelerate queries about whether two geometric entities overlap (used for culling, intersection testing in ray tracing, collision detection, …)
- Usually hierarchical data structures are used: each node defines a volume in space which, in turn contain its own children nodes: $\mathcal{O}(n) \rightarrow \mathcal{O}(\log n)$ for queries.
- Construction times for acceleration spatial data structures can be a problem, usually mitigated by performing:
  - Lazy evaluation
  - Incremental updates

Spatial Data Structures

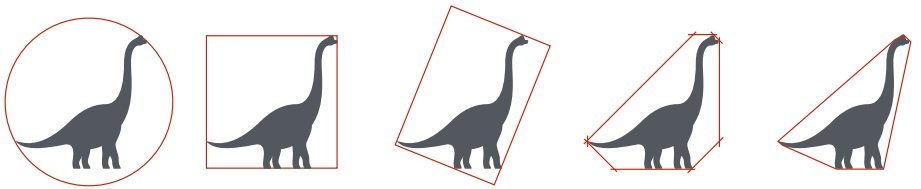
- Non Space Subdivision structures (enclose objects):
  - Bounding Volume Hierarchies (BVH) - AABB, *k*-DOP, OBB, …
- Space Subdivision structures (subdivides scene space):
  - Binary Space Partition Trees (BSP Trees),
  - Quadtrees and Octrees
  - Regular Grids

Bounding Volume Hierarchies (BVH)

- A (invisible) volume that encloses a set of objects
- The bounding volume shape must be easier to test comparing to its children
  - Bounding Spheres
  - Axis Aligned Bounding Boxes (AABB)
  - Oriented Bounding Boxes (OBB)
  - k-Discrete Oriented Polytopes (k-DOP)
- Leaf nodes contain the actual geometries
- For k-ary balanced trees, with n nodes in total, depth is $\approx \log_k n$
- Used in Hierarchical View Frustum Culling to speed-up rendering (later)



Bounding Volume Hierarchy

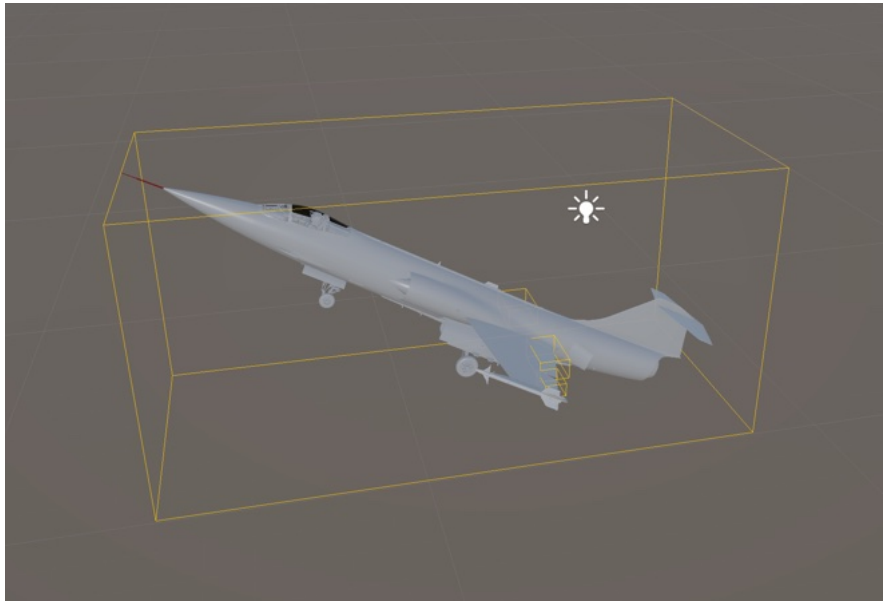


Bounding Volume Hierarchies (BVH)

- Operations needed:
  - Create a volume from a set of points
  - Volume approximation precision
  - Intersect with other bounding volume
  - Intersect with line
  - Point location
  - Rigid body object transformations
  - Union of two volumes

BVH: AABB

- Axis Aligned Bounding Box
- For each axis, compute $[min, max]$
- Simple volume creation
- Simple intersection tests
- Non straight-forward updates when children get updated
- Poor approximation



AABB

Data structure

```
public struct AABB : IBoundingVolume
{
    public Vector3 Min; // Corner with smallest x, y, z
    public Vector3 Max; // Corner with largest  x, y, z

    public AABB(Vector3 min, Vector3 max)
    {
        Min = min;
        Max = max;
    }
    // ...
}
```

Creation

```
    private static AABB BuildFromPointList(List<Vector3> pts)
    {
        float minX = float.MaxValue, minY = float.MaxValue, minZ = float.MaxValue;
        float maxX = float.MinValue, maxY = float.MinValue, maxZ = float.MinValue;

        for (int i = 0; i < pts.Count; i++)
        {
            Vector3 p = pts[i];
            if (p.X < minX) minX = p.X;
            if (p.Y < minY) minY = p.Y;
            if (p.Z < minZ) minZ = p.Z;
            if (p.X > maxX) maxX = p.X;
            if (p.Y > maxY) maxY = p.Y;
            if (p.Z > maxZ) maxZ = p.Z;
        }

        return new AABB(new Vector3(minX, minY, minZ), new Vector3(maxX, maxY, maxZ));
    }
```

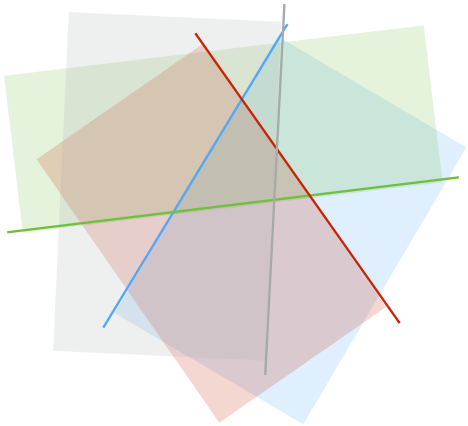
Point inside

```
    public bool Contains(Vector3 p)
    {
        return p.X >= Min.X && p.X <= Max.X &&
               p.Y >= Min.Y && p.Y <= Max.Y &&
               p.Z >= Min.Z && p.Z <= Max.Z;
    }
```

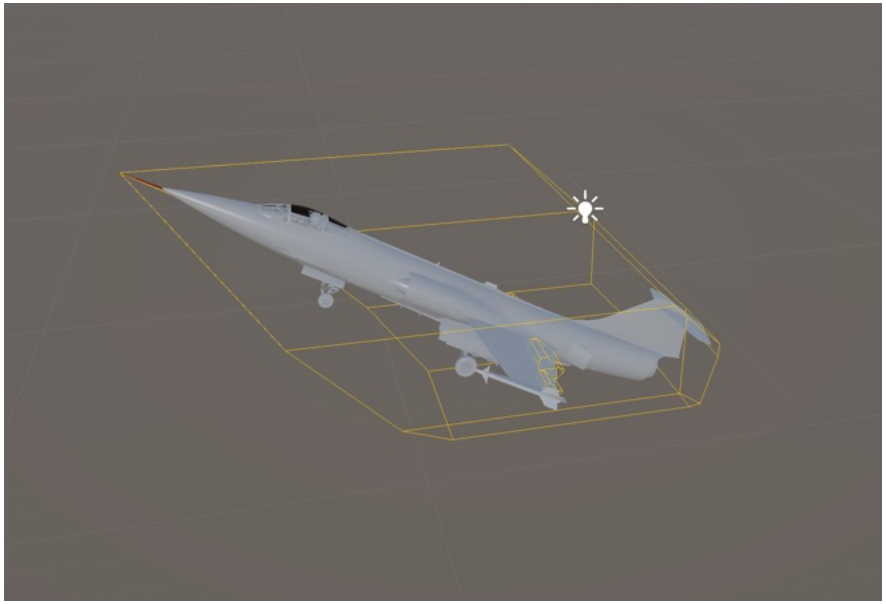
Intersection

```
    public bool Intersects(AABB other)
    {
        // Separated on ANY axis  →  no intersection
        if (Max.X < other.Min.X || Min.X > other.Max.X) return false;
        if (Max.Y < other.Min.Y || Min.Y > other.Max.Y) return false;
        if (Max.Z < other.Min.Z || Min.Z > other.Max.Z) return false;
        return true;
    }
```

BVH: k-DOP



- DOP=Discrete Oriented Polytope (Polygon, Polyhedron, …)
- Intersection of k half spaces (convex hull is the smallest k-DOP)
- Usually defined by choosing $k/2$ directions and computing the $\{min, max\}$ values for each direction
- AABB is a 6-DOP with directions determined by the coordinate axis



BVH: k-DOP

Data structure

```
public struct KDOP : IBoundingVolume
{
    public Vector3[] Directions;
    public float[] MinDistances;
    public float[] MaxDistances;

    public KDOP(Vector3[] directions, float[] minDistances, float[] maxDistances)
    {
        Directions = directions;
        MinDistances = minDistances;
        MaxDistances = maxDistances;
   }
    //...
}
```

Point inside

```
    public bool Contains(Vector3 point)
    {
        if (Directions == null || MinDistances == null || MaxDistances == null)
            return false;

        for (int d = 0; d < Directions.Length; d++)
        {
            float proj = Vector3.Dot(point, Directions[d]);
            if (proj < MinDistances[d] || proj > MaxDistances[d])
                return false;
        }

        return true;
    }
```

Intersection

```
    public bool Intersects(KDOP other)
    {

        for (int d = 0; d < Directions.Length; d++)
        {
            if (MaxDistances[d] < other.MinDistances[d] ||
                MinDistances[d] > other.MaxDistances[d])
                return false;
        }

        return true;
    }
```

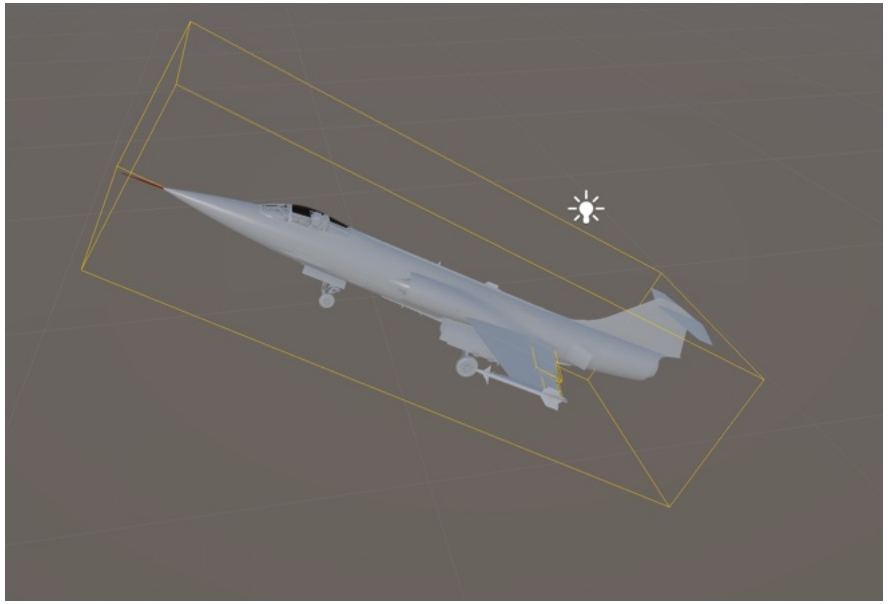
Creation

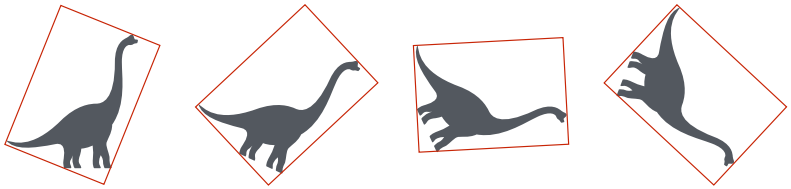
```
private static KDOP BuildFromPointAndDirectionLists(List<Vector3> pts, List<Vector3> dirs)
{
    Vector3[] normalized = new Vector3[dirs.Count];
    float[] mins = new float[dirs.Count];
    float[] maxs = new float[dirs.Count];

    for (int d = 0; d < dirs.Count; d++)
    {
        normalized[d] = Vector3.Normalize(dirs[d]);
        mins[d] = float.MaxValue;
        maxs[d] = float.MinValue;
    }

    for (int p = 0; p < pts.Count; p++)
    {
        Vector3 point = pts[p];
        for (int d = 0; d < normalized.Length; d++)
        {
            float proj = Vector3.Dot(point, normalized[d]);
            if (proj < mins[d]) mins[d] = proj;
            if (proj > maxs[d]) maxs[d] = proj;
        }
    }

    return new KDOP(normalized, mins, maxs);
}
```


Fernando Birra
Rui Nóbrega

BVH: OBB

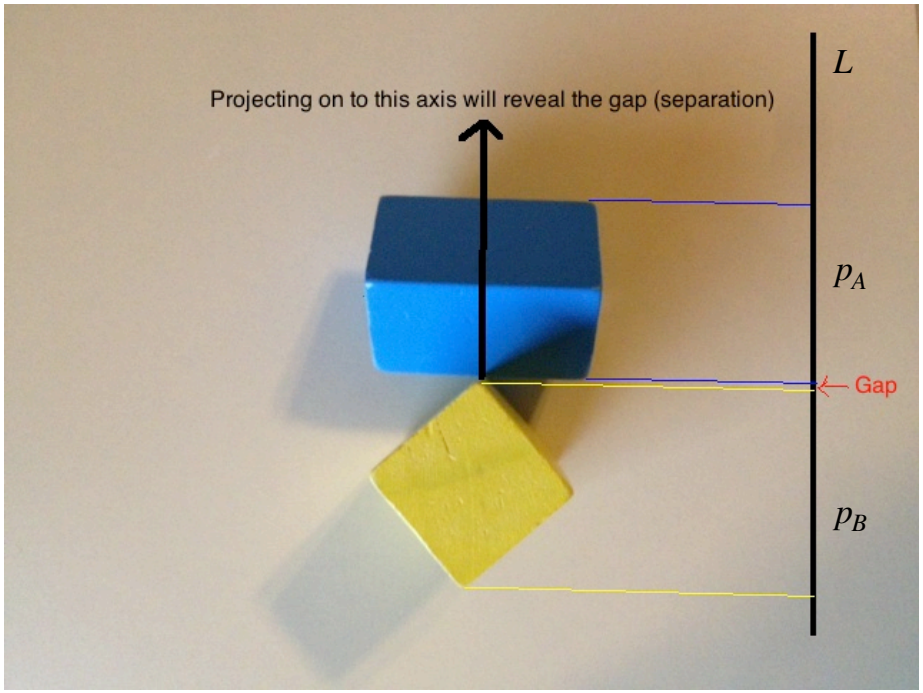
- Oriented Bounding Box
  (Rotated Axis Aligned Bouding Box)
- Better approximation to enclosed object
- More complex intersection tests (SAT)
- Easier transformations (at leaf nodes)





BVH: OBB

- Intersection of two OBB →use “Separating Axis Theorem”
- Two objects A and B do not intersect if there is some line L, such that projections of A and B onto line L do not intersect
- Direction of L is defined using polygons of bounding volume of objects A and B and using the cross product of edges from A and B



Source: gamedev.stackexchange.com

BVH: OBB

Data structure

```
public struct OBB : IBoundingVolume
{
    public Vector3 Center;
    public Vector3 AxisX;
    public Vector3 AxisY;
    public Vector3 AxisZ;
    public Vector3 HalfExtents;

    public OBB(Vector3 center, Vector3 axisX, Vector3 axisY, Vector3 axisZ,
Vector3 halfExtents)
    {
        Center = center;
        AxisX = Vector3.Normalize(axisX);
        AxisY = Vector3.Normalize(axisY);
        AxisZ = Vector3.Normalize(axisZ);
        HalfExtents = new Vector3(Math.Abs(halfExtents.X),
Math.Abs(halfExtents.Y), Math.Abs(halfExtents.Z));
    }
    //...
}
```

Point inside

```
    public bool Contains(Vector3 point)
    {
        Vector3 delta = point - Center;
        float localX = Vector3.Dot(delta, AxisX);
        float localY = Vector3.Dot(delta, AxisY);
        float localZ = Vector3.Dot(delta, AxisZ);

        return Math.Abs(localX) <= HalfExtents.X &&
               Math.Abs(localY) <= HalfExtents.Y &&
               Math.Abs(localZ) <= HalfExtents.Z;
    }
```

Creation

```
private static OBB BuildFromPointList(List<Vector3> pts)
{
    Vector3 mean = ComputeMean(pts);
    ComputeCovariance(pts, mean, out float xx, out float xy, out float xz,
                                                out float yy, out float yz,
                                                              out float zz);

    Vector3 axisX = PowerIteration(xx, xy, xz, yy, yz, zz, new Vector3(1f, 1f, 1f));
    if (axisX.LengthSquared <= 1e-8f) axisX = new Vector3(1f, 0f, 0f);
    axisX = Vector3.Normalize(axisX);

    Vector3 axisY = PowerIteration(xx, xy, xz, yy, yz, zz, new Vector3(0f, 1f, 0f), axisX);
    axisY -= axisX * Vector3.Dot(axisY, axisX);
    if (axisY.LengthSquared <= 1e-8f) axisY = AnyPerpendicular(axisX);
    axisY = Vector3.Normalize(axisY);

    Vector3 axisZ = Vector3.Cross(axisX, axisY);
    if (axisZ.LengthSquared <= 1e-8f) axisZ = AnyPerpendicular(axisX);
    axisZ = Vector3.Normalize(axisZ);
    axisY = Vector3.Normalize(Vector3.Cross(axisZ, axisX));

    float minX = float.MaxValue, minY = float.MaxValue, minZ = float.MaxValue;
    float maxX = float.MinValue, maxY = float.MinValue, maxZ = float.MinValue;

    for (int i = 0; i < pts.Count; i++)
    {
        Vector3 delta = pts[i] - mean;
        float localX = Vector3.Dot(delta, axisX);
        float localY = Vector3.Dot(delta, axisY);
        float localZ = Vector3.Dot(delta, axisZ);

        if (localX < minX) minX = localX; if (localY < minY) minY = localY;
        if (localZ < minZ) minZ = localZ; if (localX > maxX) maxX = localX;
        if (localY > maxY) maxY = localY; if (localZ > maxZ) maxZ = localZ;
    }

    Vector3 center = mean
        + axisX * ((minX + maxX) * 0.5f)
        + axisY * ((minY + maxY) * 0.5f)
        + axisZ * ((minZ + maxZ) * 0.5f);

    Vector3 halfExtents = new Vector3(
        (maxX - minX) * 0.5f,
        (maxY - minY) * 0.5f,
        (maxZ - minZ) * 0.5f);

    return new OBB(center, axisX, axisY, axisZ, halfExtents);
}
```

Intersection

```
public bool Intersects(OBB other)
{
    const float epsilon = 1e-6f;

    Vector3[] aAxes = { AxisX, AxisY, AxisZ };
    Vector3[] bAxes = { other.AxisX, other.AxisY, other.AxisZ };
    float[] aExtents = { HalfExtents.X, HalfExtents.Y, HalfExtents.Z };
    float[] bExtents = { other.HalfExtents.X, other.HalfExtents.Y, other.HalfExtents.Z };

    float[,] rotation = new float[3, 3];
    float[,] absRotation = new float[3, 3];

    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 3; j++)
        {
            rotation[i, j] = Vector3.Dot(aAxes[i], bAxes[j]);
            absRotation[i, j] = Math.Abs(rotation[i, j]) + epsilon;
        }
    }

    Vector3 translation = other.Center - Center;
    float[] t =
    {
        Vector3.Dot(translation, aAxes[0]),
        Vector3.Dot(translation, aAxes[1]),
        Vector3.Dot(translation, aAxes[2])
    };

    for (int i = 0; i < 3; i++)
    {
        float radiusA = aExtents[i];
        float radiusB = bExtents[0] * absRotation[i, 0] + bExtents[1] * absRotation[i, 1] + bExtents[2] * absRotation[i, 2];
        if (Math.Abs(t[i]) > radiusA + radiusB)
            return false;
    }

    for (int j = 0; j < 3; j++)
    {
        float radiusA = aExtents[0] * absRotation[0, j] + aExtents[1] * absRotation[1, j] + aExtents[2] * absRotation[2, j];
        float radiusB = bExtents[j];
        float projectedTranslation = Math.Abs(t[0] * rotation[0, j] + t[1] * rotation[1, j] + t[2] * rotation[2, j]);
        if (projectedTranslation > radiusA + radiusB)
            return false;
    }

    if (Math.Abs(t[2] * rotation[1, 0] - t[1] * rotation[2, 0]) > aExtents[1] * absRotation[2, 0] + aExtents[2] * absRotation[1, 0] + bExtents[1] * absRotation[0, 2] + bExtents[2] * absRotation[0, 1]) return false;
    if (Math.Abs(t[2] * rotation[1, 1] - t[1] * rotation[2, 1]) > aExtents[1] * absRotation[2, 1] + aExtents[2] * absRotation[1, 1] + bExtents[0] * absRotation[0, 2] + bExtents[2] * absRotation[0, 0]) return false;
    if (Math.Abs(t[2] * rotation[1, 2] - t[1] * rotation[2, 2]) > aExtents[1] * absRotation[2, 2] + aExtents[2] * absRotation[1, 2] + bExtents[0] * absRotation[0, 1] + bExtents[1] * absRotation[0, 0]) return false;

    if (Math.Abs(t[0] * rotation[2, 0] - t[2] * rotation[0, 0]) > aExtents[0] * absRotation[2, 0] + aExtents[2] * absRotation[0, 0] + bExtents[1] * absRotation[1, 2] + bExtents[2] * absRotation[1, 1]) return false;
    if (Math.Abs(t[0] * rotation[2, 1] - t[2] * rotation[0, 1]) > aExtents[0] * absRotation[2, 1] + aExtents[2] * absRotation[0, 1] + bExtents[0] * absRotation[1, 2] + bExtents[2] * absRotation[1, 0]) return false;
    if (Math.Abs(t[0] * rotation[2, 2] - t[2] * rotation[0, 2]) > aExtents[0] * absRotation[2, 2] + aExtents[2] * absRotation[0, 2] + bExtents[0] * absRotation[1, 1] + bExtents[1] * absRotation[1, 0]) return false;

    if (Math.Abs(t[1] * rotation[0, 0] - t[0] * rotation[1, 0]) > aExtents[0] * absRotation[1, 0] + aExtents[1] * absRotation[0, 0] + bExtents[1] * absRotation[2, 2] + bExtents[2] * absRotation[2, 1]) return false;
    if (Math.Abs(t[1] * rotation[0, 1] - t[0] * rotation[1, 1]) > aExtents[0] * absRotation[1, 1] + aExtents[1] * absRotation[0, 1] + bExtents[0] * absRotation[2, 2] + bExtents[2] * absRotation[2, 0]) return false;
    if (Math.Abs(t[1] * rotation[0, 2] - t[0] * rotation[1, 2]) > aExtents[0] * absRotation[1, 2] + aExtents[1] * absRotation[0, 2] + bExtents[0] * absRotation[2, 1] + bExtents[1] * absRotation[2, 0]) return false;

    return true;
}
```

BVH Construction

Bottom-up

- Starting with separate objects
- Create bonding volume for each object (leaves)
- Group bounding volumes recursively into bigger volumes
- Repeat until single root bounding volume

BVH Construction

Top-Down

- Create bonding volume for the whole scene
- Split volumes recursively in k parts each, by splitting the primitives/objects into k sets.
- Finish when current volume has an primitive/object count less than given threshold

BVH Updates

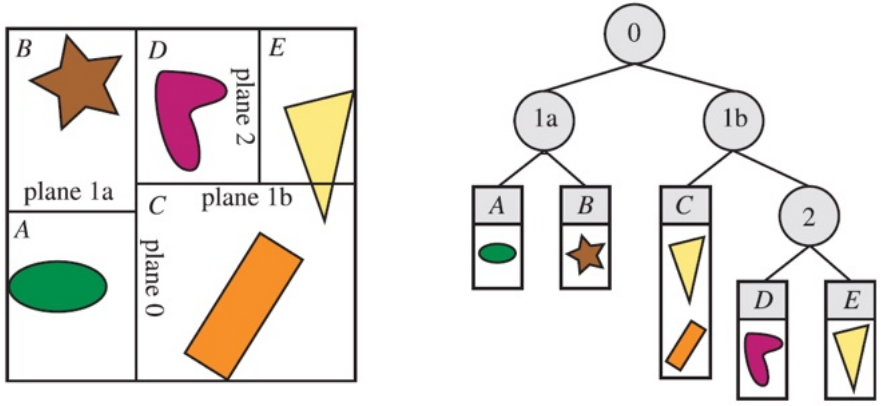
- In dynamic scenes, BVH need to be updated:
  - When an object moves, if its parent BV still bounds it no need to update
  - If not:
    - Refit bounding volume and perform upwards propagation of updates
    - Remove the object, recompute parent BV, recursively insert object into BVH again
  - Whichever strategy is used to update it, BVH degrades, becomes unbalanced
- Another alternative is to use temporal bounding volumes:
  - Enlarge bounding volume of object with its expected movement over a period of time
  - Easier for kinematic simulations harder for physical ones

BSP Trees

- Two major variants
  - Axis aligned
  - Polygon aligned
- Used for sorting: front-to-back (minimize pixel overdraw) or back-to-front (draw transparent objects), view frustum culling, occlusion culling or ray tracing tests acceleration

Axis-Aligned BSP Trees

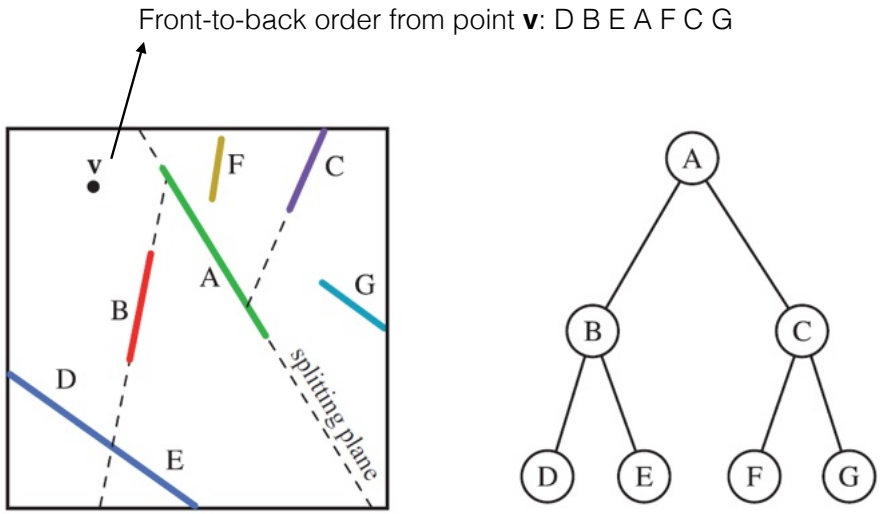
- Enclose whole scene in an Axis Aligned Bounding Box (AABB)
- Recursively subdivide this box into smaller boxes
  - One axis of the box is chosen (greatest extent, round robin, highest variance, best quality given by some heuristics such as SAH-Surface Area Heuristic)
  - A perpendicular planes is inserted
    - Uniform spacing → easier to create
    - Varying position(s) → more balanced
  - When an object is intersected by a plane it can be stored at the node, made a member of both neighboring nodes, or separated by the plane into two different objects
  - Repeat until number of objects for the node is below a given threshold.
- Provide **rough** sorting of objects (back-to-front or front-to-back)



Axis Aligned Binary Space Partition Tree

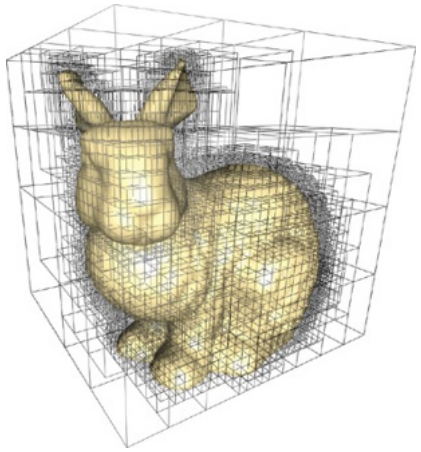
Polygon-Aligned BSP Trees

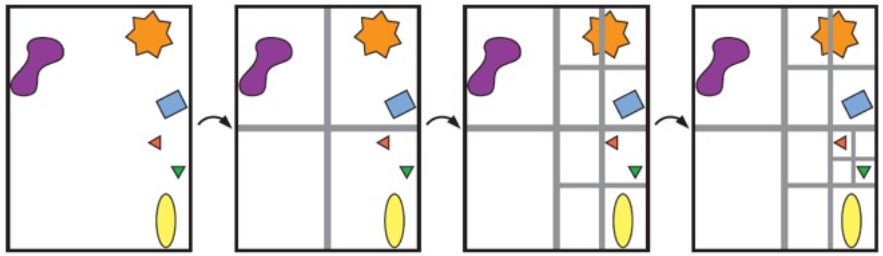
- Adequate for static or rigid geometry
- Popular in games like Doom (1993 - No Z-Buffer hardware, used Painter's algorithm)
- Repeat recursively until all polygons are treated:
  - At each node, a polygon is chosen, the plane where it lies splits space in two halves
  - Polygons cut by the place are split into two parts, one for each child node
- Time consuming process
- Provide **exact** sorting of objects (back-to-front or front-to-back)



Octrees

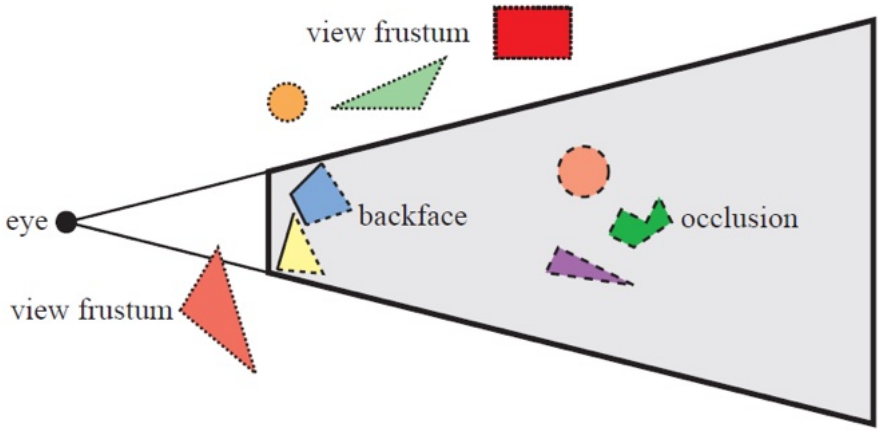
- Similar to Axis Aligned BSP trees, but the box is split by the box center along the three axis, creating 8 new boxes
- A quad tree is the equivalent of an octree for the 2D case
- Start with a minimal AABB
- For each node, repeat split in 8 until stop criteria
  - Maximum depth reached
  - Threshold on the number of primitives
  - Objects intersected by more than 1 sub volume can:
    - Be placed in all intersected sub volumes
    - Choose the smallest that fully contains the object
    - Use loose octrees [https://www.tulrich.com/geekstuff/partitioning.html]





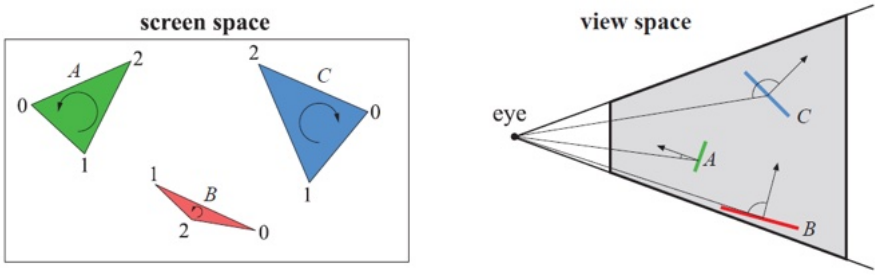
Culling Techniques

- Not all components of a scene will make their way to the final frame
  - Some objects are outside the view frustum → view frustum culling
  - Some objects are occluded by other → occlusion culling
  - Some polygons/triangles are occluded by others from the same object → backface culling (polyhedra)

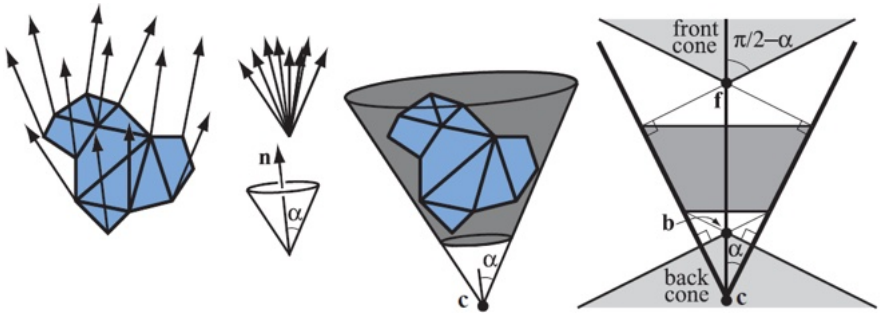


Backface Culling

- For polyhedra, approximately half of the faces/triangles are occluded by the other half
- When we look towards a specific direction there is a front facing face and a back facing face (that doesn't need to be rendered)
- Test is performed for **individual faces**
- Clustered backface cullingcan be used to avoid sending **groups of faces** to the 3D API pipeline (example: truncated normal cones)



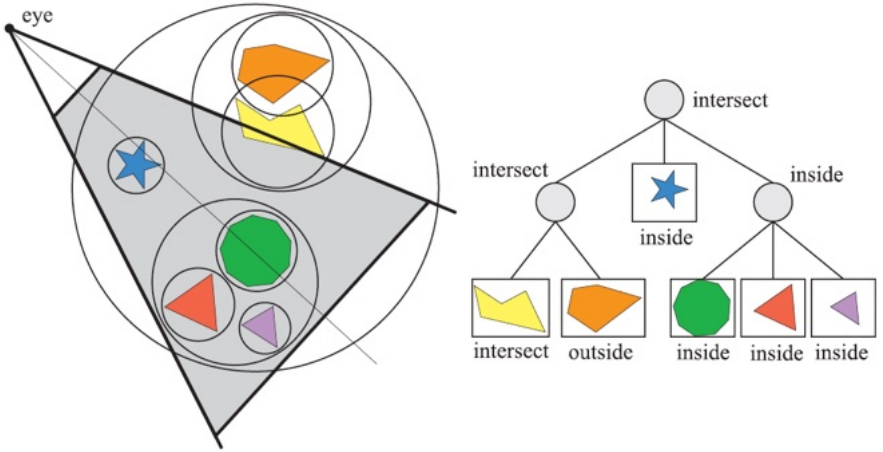
Backface culling



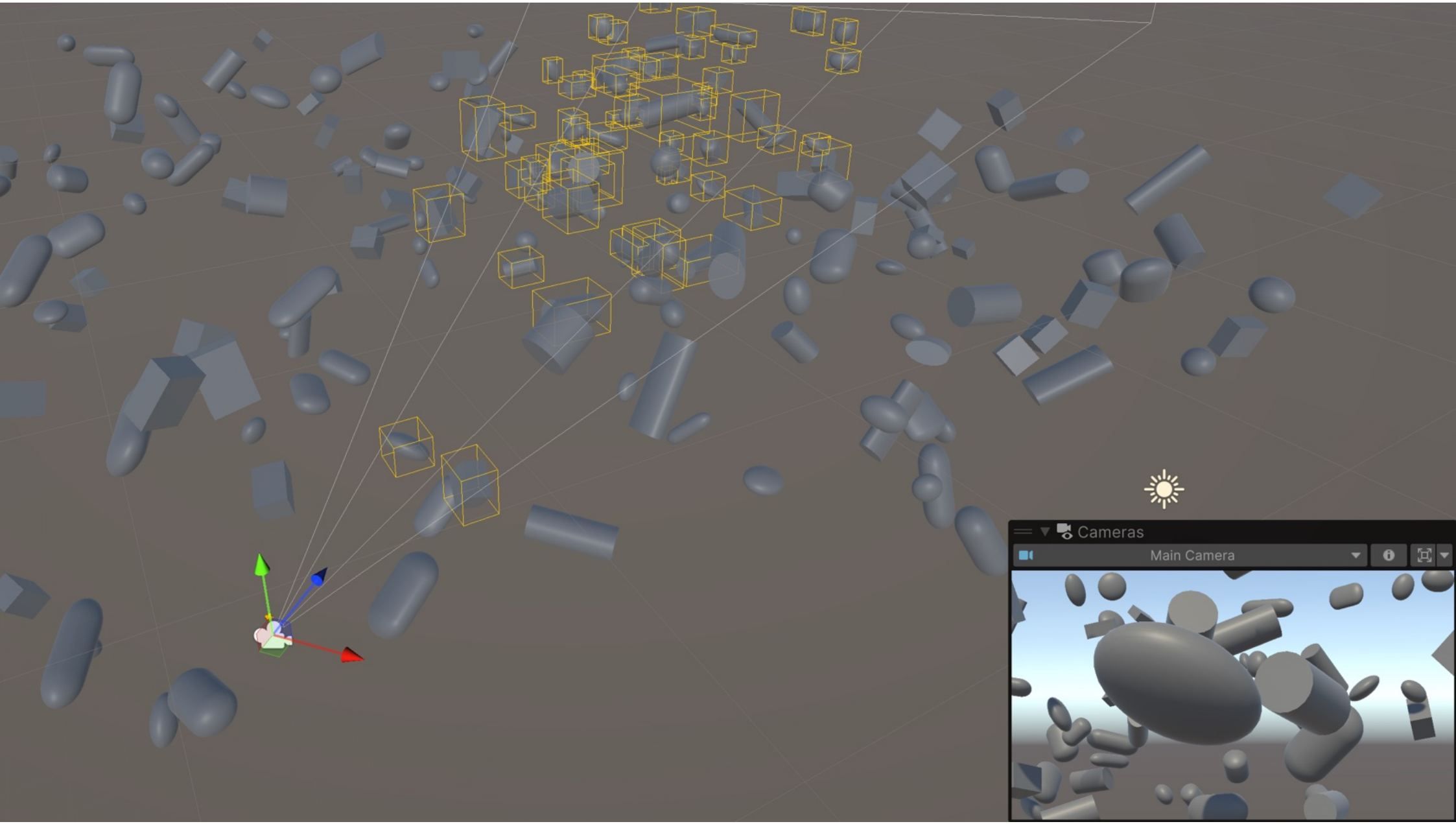
Truncated normal cones

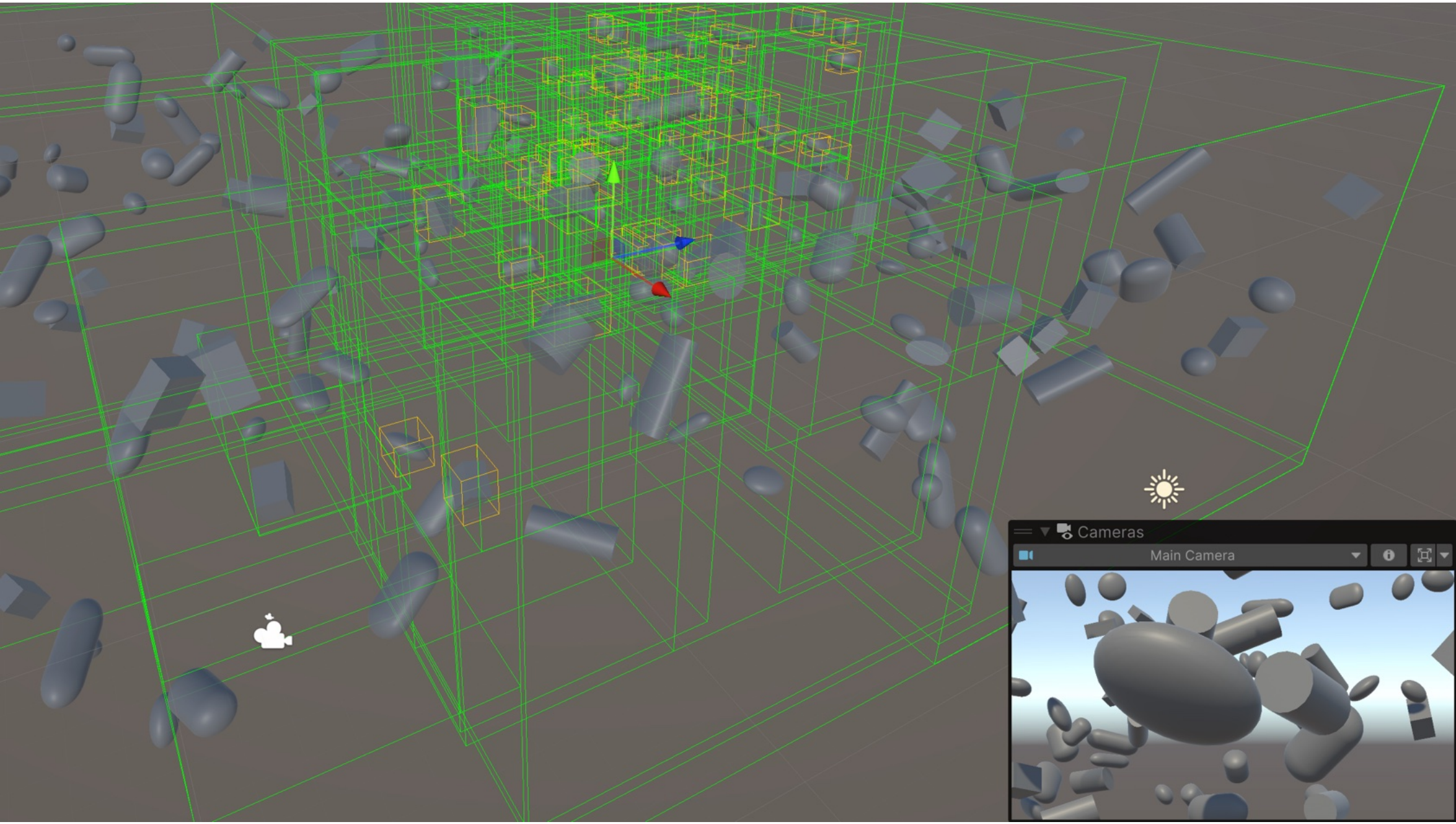
View Frustum Culling

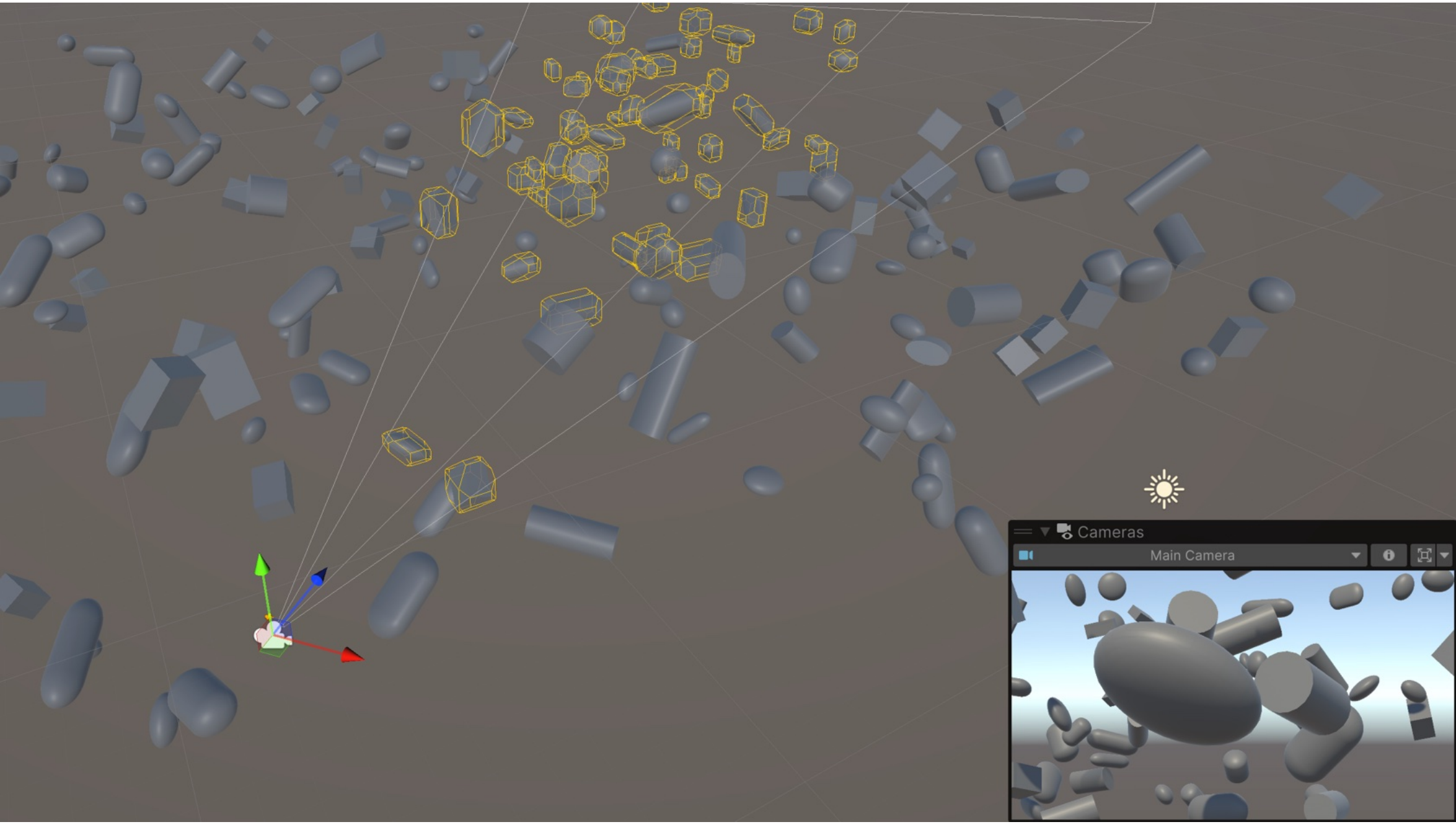
- Only primitives that are entirely or partially inside the view volume need to be rendered
- One possibility is to test if bounding volume of primitive is inside the view frustum
- A better approach is to use a spatial data structure and intersect the view volume with the its nodes:
  - If a node is **completely outside** the frustum
    - stop recursion and prune the tree
  - If a node is **completely inside** the frustum
    - stop recursion and pass all primitives at the leaves of the subtree to the next stage
  - If a node is partially inside the frustum
    - If it is a leaf pass the primitives to the next stage
    - If it is an internal node recursively process its children
  - We may keep track at each node of which planes of the view frustum the volume is inside (6-bit mask). Children need only to retest those planes…

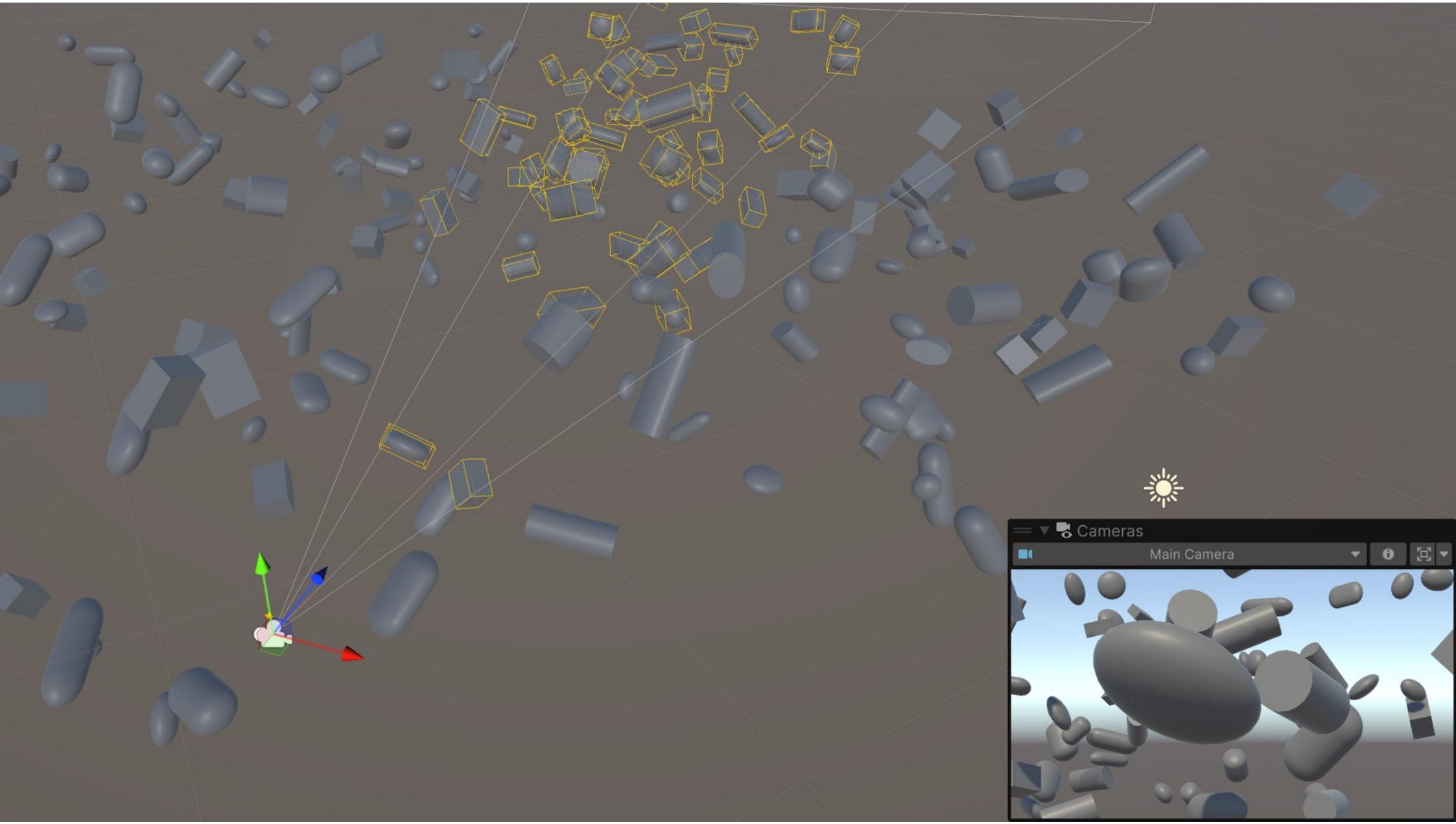


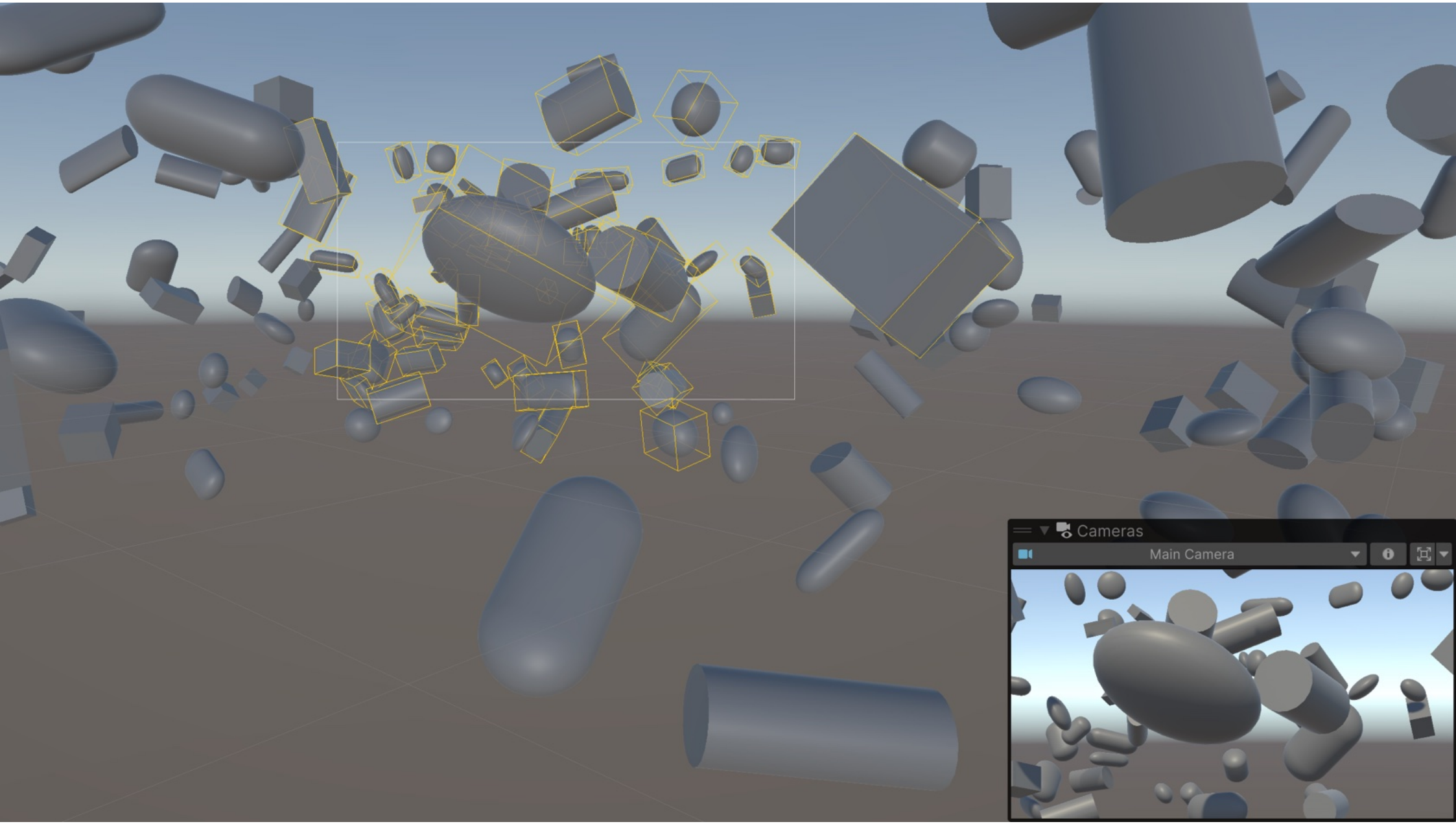
View Frustum Culling


Cameras
Main Camera


Cameras
Main Camera

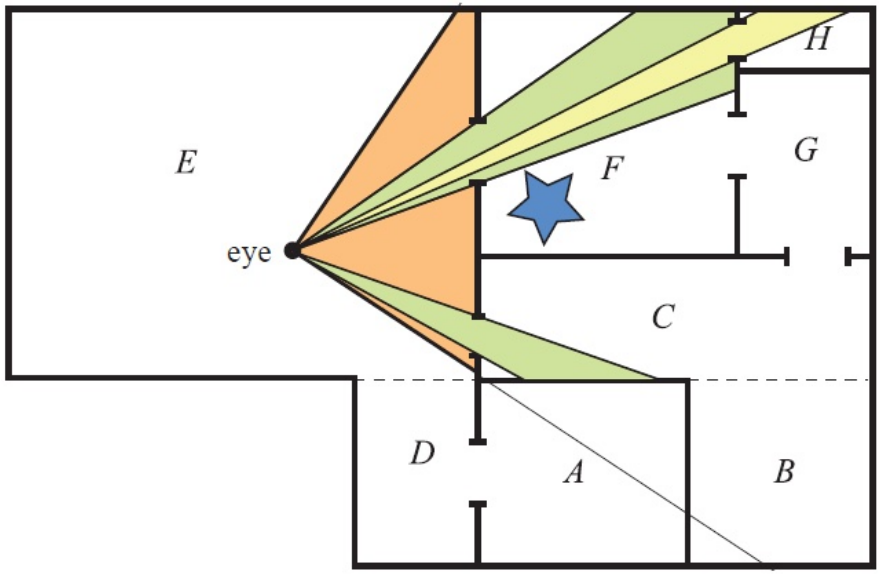

Cameras
Main Camera


Cameras
Main Camera


Cameras
Main Camera

Portal Culling

- Adequate for architectural models with doors and windows
- Can be seen as an extension of view frustum, further restricted to the boundaries of doors/windows
- Each room is a adjacent cells
- When rendering the contents of a cell, cull using the view frustum as restricted by the edges of the portal
- Needs to be applied recursively: When rendering cell F culled by the portal between E and F, we can see the portal leading to H. Thus, when rendering contents of H we need to use the frustum defined by the portal between F and H
- Every frustum has the eye as its apex

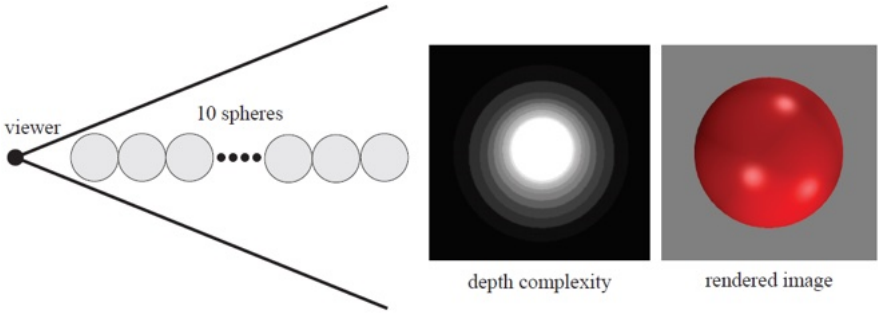


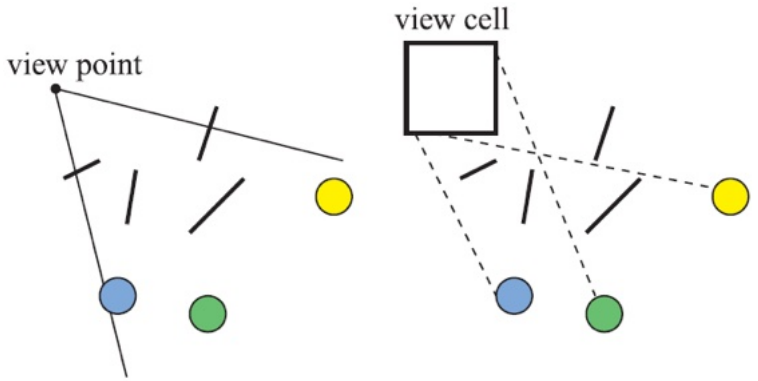
Portal Culling (with mirrors)

https://luebke.us/publications/pdf/portals.pdf

Occlusion Culling

- Visibility may be solved via z-buffer (simple and brute force approach)
- Occlusion Culling tries to cull away objects hidden by others
- View point (valid for a single view point) based vs view cell based (valid while viewpoint inside cell)
- Image space vs Object Space algorithms

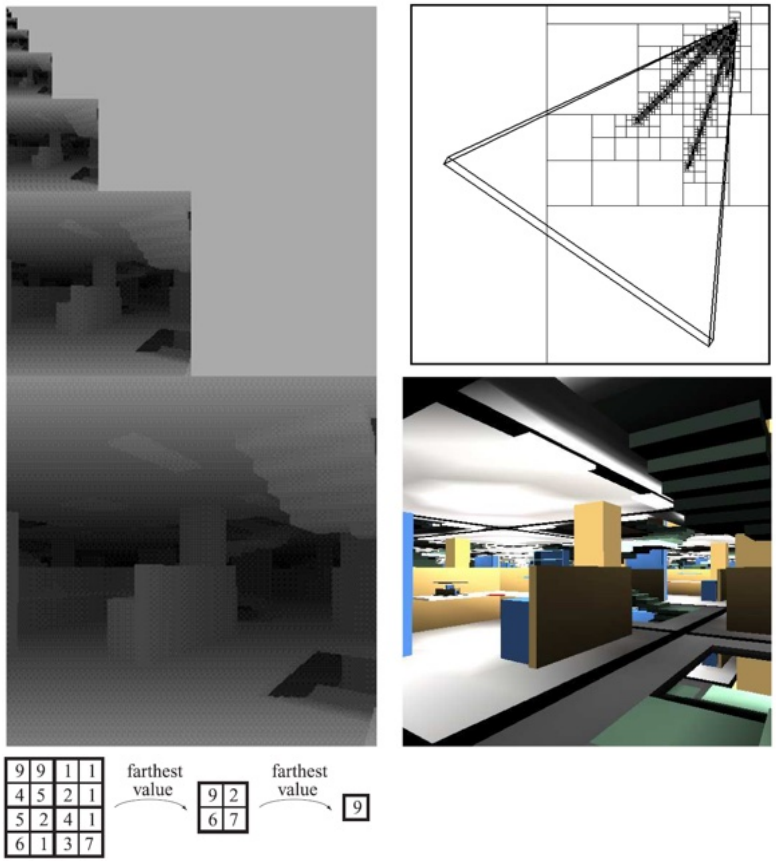




Hierarchical Z-Buffer

[https://tr.soe.ucsc.edu/sites/default/files/technical-reports/UCSC-CRL-95-27.pdf]

- Octree with scene objects
- Z-Buffer Pyramid (coarser level stores furthest value from 2x2 grid from immediate finer level)
- Traverse Octree nodes in rough front-to-back order:
  - Project bounding box of node onto screen measure its width w and height h
  - Choose z-pyramid level: $level = floor(\log_2(\max(w, h)))$
  - Read (at most 2x2 values) and find z_{max}
  - Compute the z-value for nearest corner of the node: $z_{nearest}$
    - $z_{nearest} > z_{far} \Rightarrow$ occluded (discard node contents)
    - Otherwise: recurse into node's children of draw geometry if at leaf.

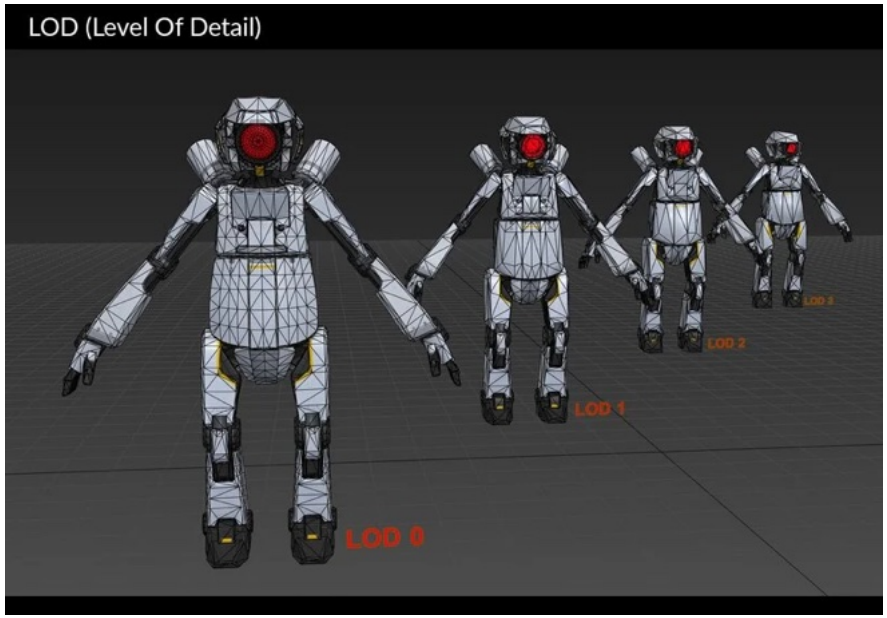


Occlusion Queries

- Modern GPU support Occlusion Queries
- GPU special mode that given a set of triangles, checks that them agains the z-buffer and returns visibility information such number of passes (visible pixels) or more simply if there was at least a visible pixel
- Combined with BVH, we can check if a given bounding volume is visible while drawing objects front-to-back
- The number of visible pixels can even assist the level of detail to be used
- Acceleration results from cases where Occlusion Query returns 0 samples →no need to draw
- Occlusion Queries may introduce latency if done naively. 3D API support associating a batch rendering command with a specific query. If it passes the rendering gets performed, to avoid polling or waiting for query results.
- Check what Unity Engine is using here

Level Of Detail (LOD)

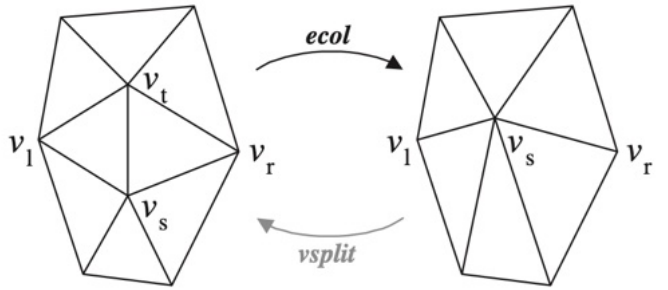
- **Rationale**: Use simpler versions of an object as its contribution to the final image decreases
- Typically **applied after culling** so that no overhead with discarded geometries
- Can be used to help achieve better frame rates in poorer hardware
- Reduce vertex processing workload
- The simpler LOD for rigid objects is often a Billboard (quads with partially transparent textures). Articulated objects may use simpler bone hierarchies.
- Fog is often used to make objects disappear at large distances without visual artifacts (reduce view frustum depth)
- Objects that can to be tessellated have embedded LOD (math described solids, Parametric surfaces, Subdivision Surfaces).



[Source]

LOD Mechanisms

- **Generation**: Different representations of a model need to be generated with different amounts of detail (**Mesh simplification** algorithms, hand crafted models, Tessellation parameter control, …)
- Selection: Select a LOD based on some criteria (projected area, range, …)
- Switching: The process of changing from one LOD to another (discrete switching, LOD blending, Alpha LODs, CLOD, Geomorphology LOD, …)



Full high-res mesh

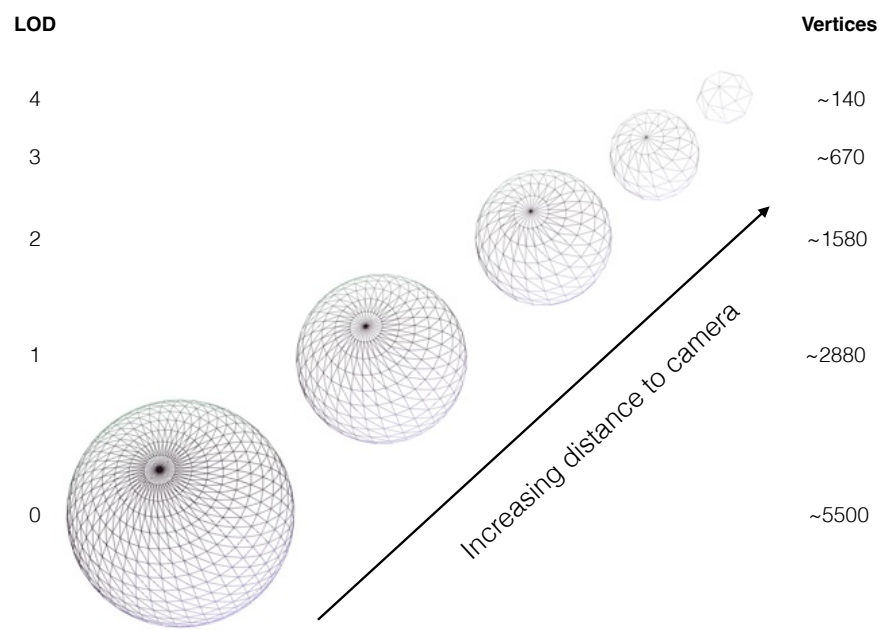
$$(\hat{M}=M^n) \xrightarrow{ecol_{n-1}} \dots \xrightarrow{ecol_1} M^1 \xrightarrow{ecol_0} M^0$$

Lowest res mesh

Progressive Meshes

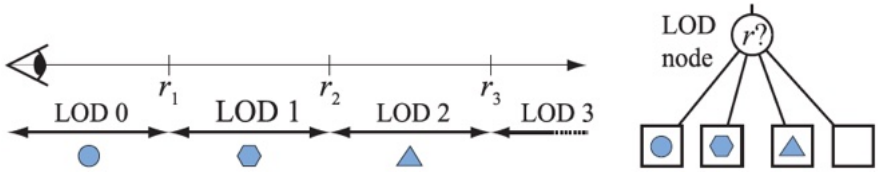
LOD Mechanisms

- **Generation**: Different representations of a model need to be generated with different amounts of detail (Mesh simplification algorithms, hand crafted models, **Tessellation** parameter **control**, …)
- Selection: Select a LOD based on some criteria (projected area, range, …)
- Switching: The process of changing from one LOD to another (discrete switching, LOD blending, Alpha LODs, CLOD, Geomorphology LOD, …)

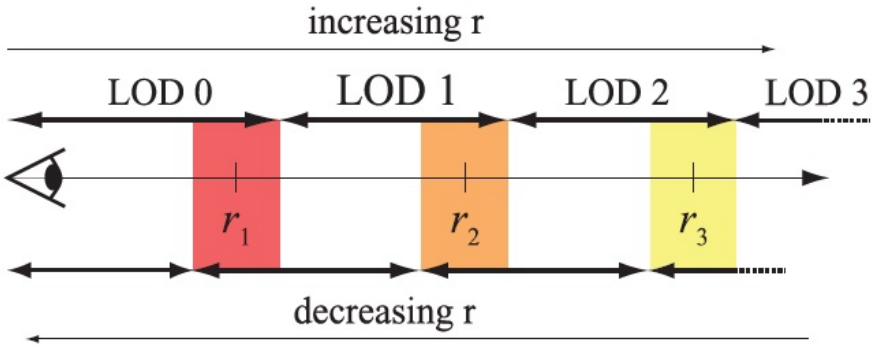


LOD Mechanisms

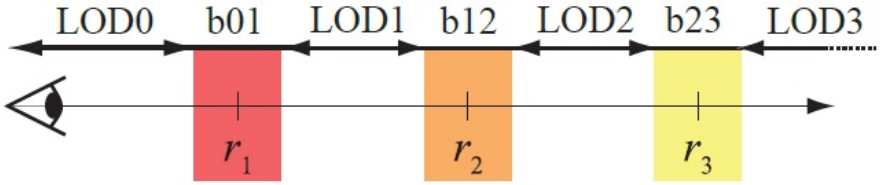
- Generation: Different representations of a model need to be generated with different amounts of detail (Mesh simplification algorithms, hand crafted models, Tessellation parameter control, …)
- **Selection**: Select a LOD based on some criteria (**range**, projected area)
- Switching: The process of changing from one LOD to another (discrete switching, LOD blending, Alpha LODs, CLOD, Geomorphology LOD, …)



Range based LOD selection



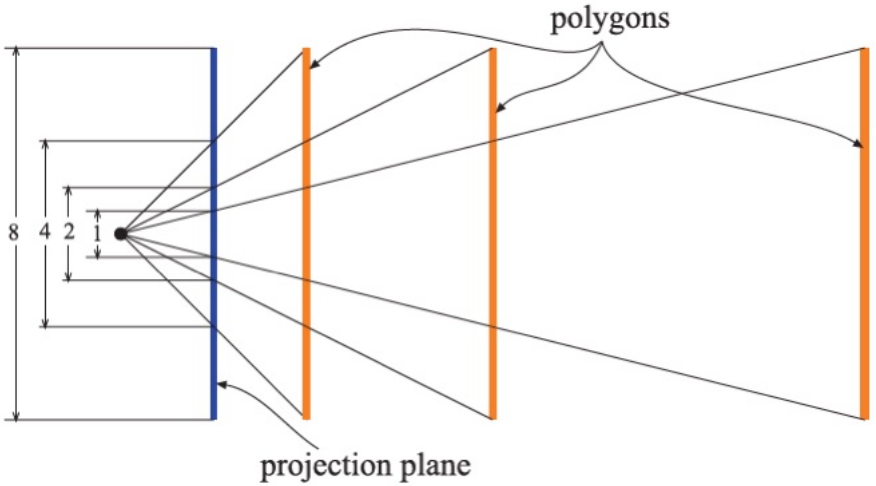
Range based LOD with hysteresis



Range based LOD with blending

LOD Mechanisms

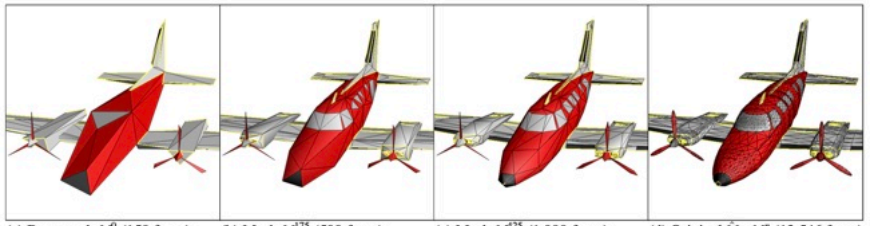
- Generation: Different representations of a model need to be generated with different amounts of detail (Mesh simplification algorithms, hand crafted models, Tessellation parameter control, …)
- **Selection**: Select a LOD based on some criteria (range, **projected area**)
- Switching: The process of changing from one LOD to another (discrete switching, LOD blending, Alpha LODs, CLOD, Geomorphology LOD, …)



Projected Area of a Thickness object: area is halved as distance is doubled

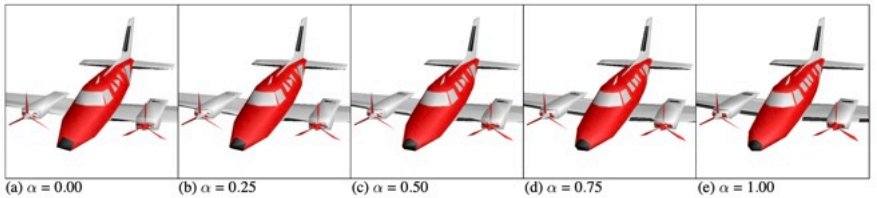
LOD Mechanisms

- Generation: Different representations of a model need to be generated with different amounts of detail (Mesh simplification algorithms, hand crafted models, Tessellation parameter control, …)
- Selection: Select a LOD based on some criteria (projected area, range, …)
- **Switching**: The process of changing from one LOD to another (discrete switching, LOD blending, Alpha LODs, CLOD, Geomorphology LOD, …)



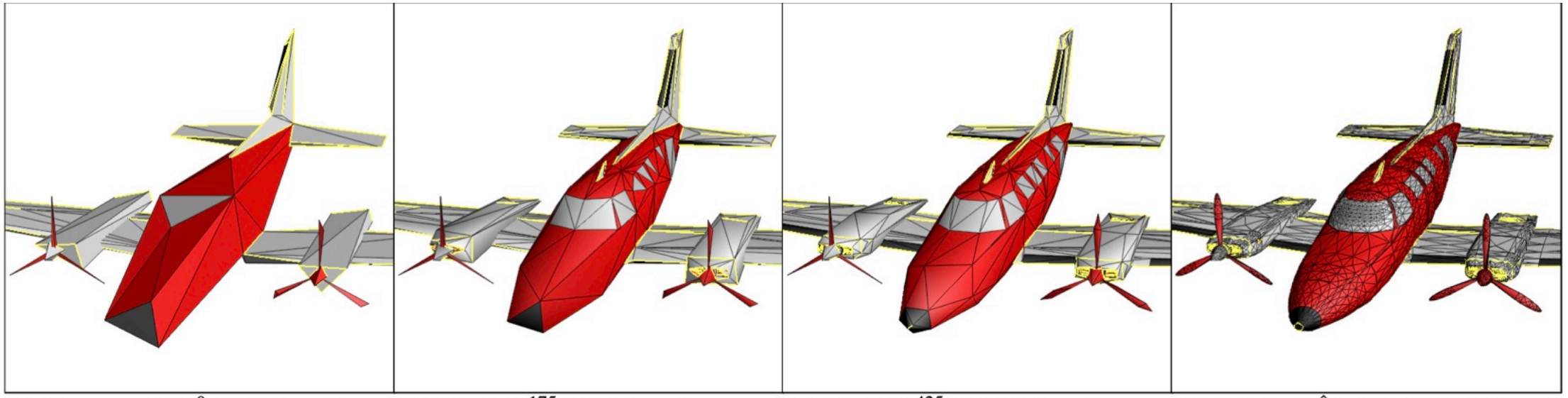
(a) Base mesh M^0 (150 faces) (b) Mesh M^{175} (500 faces) (c) Mesh M^{425} (1,000 faces) (d) Original $\tilde{M}=M^n$ (13,546 faces)

Figure 5: The PM representation of an arbitrary mesh $\tilde{M}$ captures a continuous-resolution family of approximating meshes $M^0 \ldots M^n=\tilde{M}$.



(a) $\alpha = 0.00$ (b) $\alpha = 0.25$ (c) $\alpha = 0.50$ (d) $\alpha = 0.75$ (e) $\alpha = 1.00$

Figure 6: Example of a geomorph $M^G(\alpha)$ defined between $M^G(0)\doteq M^{175}$ (with 500 faces) and $M^G(1)=M^{425}$ (with 1,000 faces).



(a) Base mesh M^0 (150 faces) (b) Mesh M^{175} (500 faces) (c) Mesh M^{425} (1,000 faces) (d) Original $\hat{M} = M^n$ (13,546 faces)

Figure 5: The PM representation of an arbitrary mesh $\hat{M}$ captures a continuous-resolution family of approximating meshes $M^0 \ldots M^n = \hat{M}$.

(a) $\alpha = 0.00$ (b) $\alpha = 0.25$ (c) $\alpha = 0.50$ (d) $\alpha = 0.75$ (e) $\alpha = 1.00$

Figure 6: Example of a geomorph $M^G(\alpha)$ defined between $M^G(0) \doteq M^{175}$ (with 500 faces) and $M^G(1) = M^{425}$ (with 1,000 faces).

LOD Related Techniques

- LOD is not just about geometry:
  - **Bump mapping** - Simulate small scale bumps using a normals texture
  - **Retopology** (Mesh simplification) - Reduce vertex, edge and face count in complex meshes while keeping its overall shape (minimize difference between both surfaces)
  - **Texture Baking** - Generate new textures to be applied to simpler/larger polygons in simplified meshes from higher resolution versions
  - **Displacement Map Baking** - Generate a texture that can be used to derive an height map of the surface to be applied to a simpler mesh (displacement is applied at pixel shader level)
  - **Simplified shading** - Use less complex shaders for faraway objects